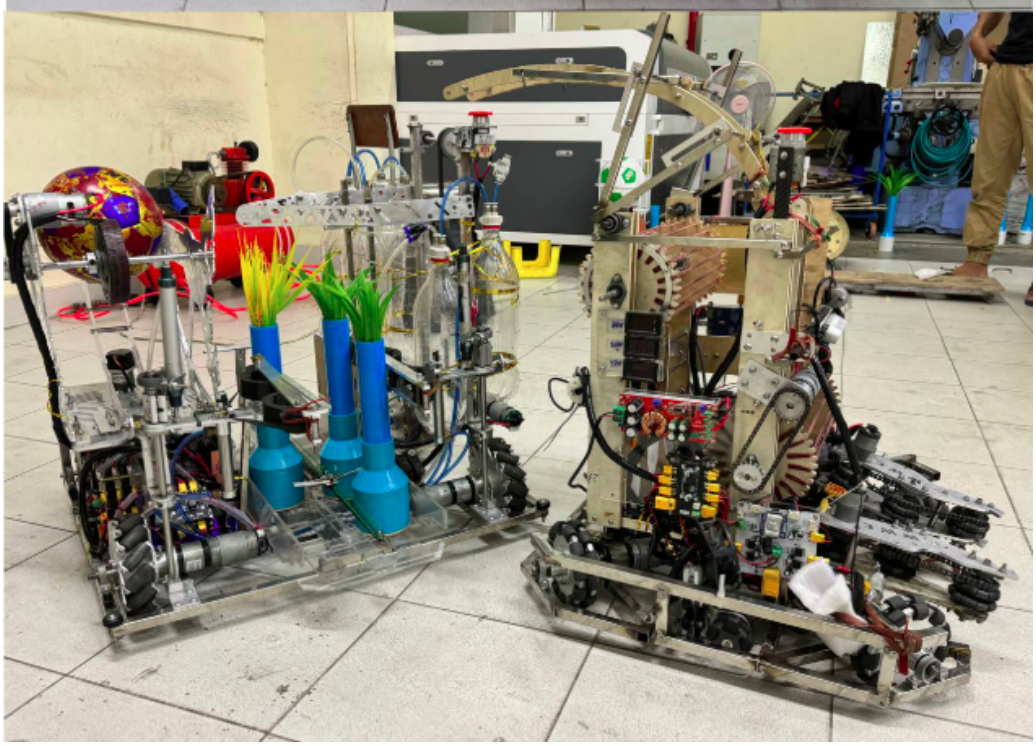
A Pratical Guide Inspired by My ROBOCON 2024 Experience

Yin Chheanyun

July 2024





Contents

1 Introduction **4**

2 System Architecture **5**
2.1 MR1 Farmer Robot System Architecture 5
2.2 MR2 Buffalo Robot System Architecture 6

3 Lecture Review **7**
3.1 Mecanum Wheel 7
3.2 Omni Wheel 10
3.3 PID CONTROLLER 11
3.3.1 PID Algorithm 12
3.3.2 Integral Windup 12
3.3.3 Low pass filter 13
3.3.4 Output Saturation 13
3.4 LQR Controller 13
3.4.1 LQR Algorithm 14
3.5 NONLINEAR MODEL PREDICTIVE CONTROL (NMPC) CONTROLLER . 15
3.6 NMPC Algorithm Discrete-time 16
3.7 Extended Kalman Filter(EKF) 18
3.7.1 Extended Kalman Filter Algorithm 19

4 Methodology **22**
4.1 Components 22
4.2 Control Speed of DC Motor using PI Controller 22
4.2.1 DC Motor Modeling 22
4.2.2 PI Tuning 25
4.3 P controller with Wheel Robot 27
4.4 MPC Controller with Wheel Robot 28

5 Result and Discussion **28**
5.1 Tuning DC Motor 24V 630RPM in MATLAB 28
5.2 Simulation LQR with Mecanum Wheel 31
5.3 Testing Farmer Robot Version 1 with STM32F407VGT6 32
5.3.1 PID Position Gain Tuning 33
5.3.2 Test Result 33
5.4 Testing Farmer Robot Version 2 with ROS2 33
5.4.1 System Nodes 34
5.4.2 PID Position Gain Tuning 34
5.4.3 Video Result 34
5.5 Testing Buffalo Robot with MPC Controller using ROS2 35
5.5.1 Reference Trajectory Generation 35
5.5.2 MPC Formulation 35
5.5.3 Control Architecture 36
5.5.4 Test Result 36
5.6 Testing Buffalo Robot with Full Concept Using PID Controller 37
5.6.1 Ball Detection and Position Estimation 37
5.6.2 Ball Collection and Silo Sorting Mechanism 37
5.6.3 State Machine Overview 38
5.6.4 Gain-Scheduled PID Control 38

5.6.5 Test Result 38

6 Conclusion **38**

List of Figures

1 Game field of ROBOCON 2024 4
2 System architecture of MR1 (Farmer Robot) 5
3 System architecture of MR2 (Buffalo Robot) 6
4 Movement and Deriction of mecanun wheel 7
5 Coordination and motion mecanum wheel 8
6 Movement and Direction of OMNI Wheel 10
7 PID Controller Block diagram 12
8 Cost curve of LQR 14
9 MPC Prdicted output 16
10 Block diagram of Model Predicted Controller 16
11 Extended Kalman Filter with Sensor (IMU, External Rotary Encoder, Wheel encoder) 18
12 Flow to use Extended Kalman Filter 19
13 Board PCB Design 22
14 Modeling of DC Motor 23
15 PI Tuning Method 1 on Speed DC Motor in Simulink 25
16 Block PID Paramter Setup 26
17 PI Tuning Method 2 on Speed DC Motor in MATLAB Script 27
18 Block diagram of Position Base with P-Controller 27
19 Block diagram of Position Base control with MPC 28
20 PID Tuner app result for DC Motor plant model ($K = 2.88$): tuned response vs. block response 29
21 Step input source: ideal reference step from 0 to 1 at $t = 1\,\mathrm{s}$ 30
22 Closed-loop motor speed response to the step input 30
23 Simulated LQR trajectory tracking on Mecanum wheel base (reference path vs. actual) 32
24 Reference path vs. actual trajectory tracked by MPC (MR2) 36

List of Tables

1 Table Parameter of Mecanum Wheel 9
2 List of Components 22
3 PID Tuner Result for DC Motor 24V 630RPM Speed Model 31
4 LQR Cost Matrices used in Simulation 31
5 Actuator and Simulation Constraints 32
6 LQR Simulation Tracking Performance 32
7 PID Position Controller Gains for MR1 V1 33
8 PID Controller Limits and Sampling Parameters 33
9 PID Position Controller Gains for MR1 V2 34
10 PID Controller Limits and Sampling Parameters 34
11 MPC Tuning Parameters for MR2 Omni Wheel 35
12 Gain-Scheduled PID Parameters by Robot State 38
13 Common PID Parameters and Output Limits 38

1 Introduction

The **ABU Robocon 2024**, themed *"Harvest Day"*, is the Asia-Pacific Robot Contest organised by the Asia-Pacific Broadcasting Union (ABU) and hosted in **Quang Ninh, Vietnam** for its 2024 edition. Team **ITC01**, representing the Institute of Technology of Cambodia (ITC), also competed under this theme in the **11th Cambodia Robocon 2024**.

The theme "Harvest Day" reflects the traditional highland practice of terraced rice farming in Vietnam. Teams must simulate the stages of the rice farming cycle: planting seedlings, harvesting paddy rice, and storing the harvested grains in silos. The game field is divided into **3 Areas**, with each robot performing a unique role within the cycle.

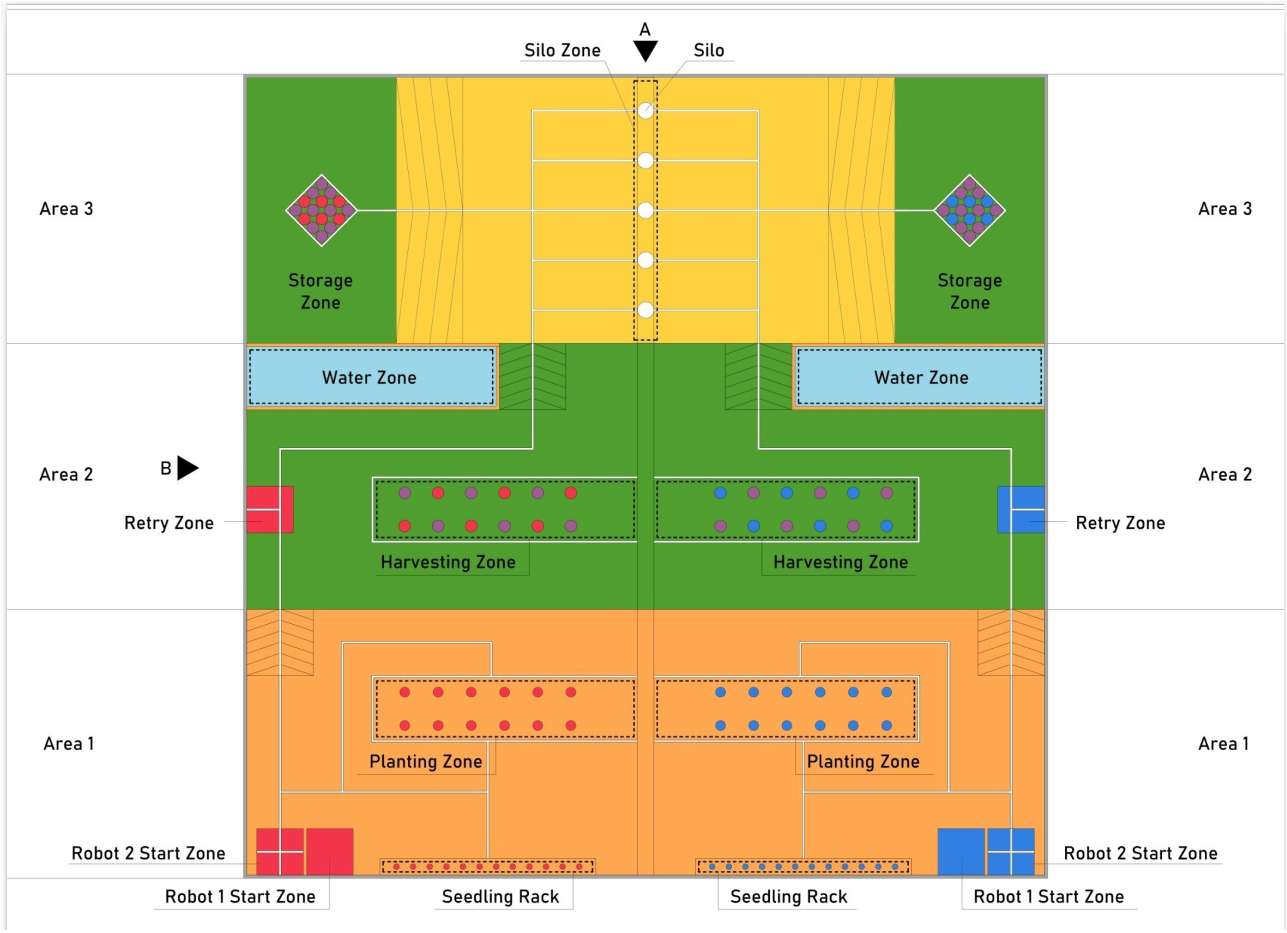


Figure 1: Game field of ROBOCON 2024

Task of Robot in each Area

- **Area 1 (Planting Zone):** Robot MR1 (Farmer Robot) plants seedlings (PVC pipes) into designated positions.
- **Area 2 (Harvest Zone):** Robot MR1 harvests paddy rice and empties grains from the field, while Robot MR2 (Buffalo Robot) also operates here to collect balls.
- **Area 3 (Storage Zone):** Robot MR2 stores the harvested paddy rice into silos, sorted according to the color of the paddy.

Robots

Each team must build 2 robots:

- **Robot 1 (MR1 – Farmer Robot):** Manually driven with assisted automation (semi-autonomous), operating in Area 1 and Area 2.
- **Robot 2 (MR2 – Buffalo Robot):** Fully autonomous, navigating and depositing balls by color using onboard perception, operating in Area 2 and Area 3.

2 System Architecture

This section describes the overall control architecture of the two robots, **MR1 (Farmer Robot)** and **MR2 (Buffalo Robot)**, including how the mechanical mechanisms, base (drive) control, and sensors are distributed across the STM32 boards and connected through CAN BUS communication.

2.1 MR1 Farmer Robot System Architecture

The Farmer Robot (MR1) architecture is divided into two main branches: **Mechanic Sim Control**, which handles the rice planting/harvesting mechanisms, and **Base Control**, which handles the drive system.

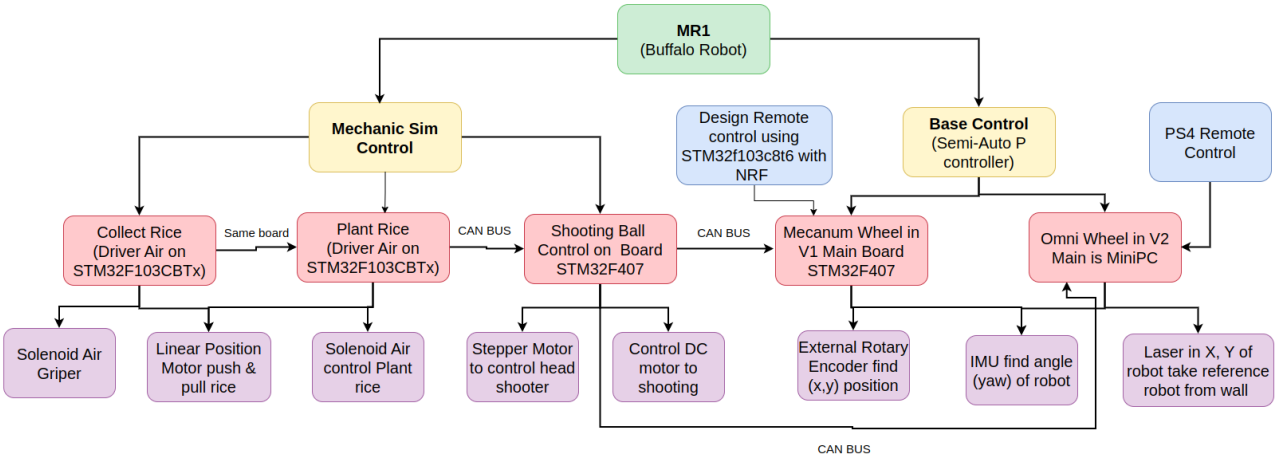


Figure 2: System architecture of MR1 (Farmer Robot)

Mechanic Sim Control

This branch controls all mechanisms responsible for rice planting and harvesting:

- **Collect Rice (STM32F103C8T6):** Controls the air solenoid driver for the rice-collecting gripper.
- **Plant Rice (STM32F103C8T6):** Shares the same board as Collect Rice; controls the linear position motor (push/pull rice mechanism) and the solenoid air driver for planting rice.
- **Shooting Ball Control (STM32F407):** Connected via CAN BUS, controls the stepper motor for aiming the shooter head and the DC motor for the shooting mechanism.

Base Control (Semi-Auto P Controller)

This branch controls the robot's drive base and remote control interface:

- **Remote Control Design:** Implemented on an STM32F103C8T6 board communicating via NRf24l01 wireless module.
- **PS4 Remote Control:** Provides direct manual control input to the drive system.
- **Mecanum Wheel (V1) – Main Board STM32F407:** Drives the 4-wheel Mecanum base, with feedback from an external rotary encoder (for X, Y position) and an IMU (for yaw angle).
- **Omni Wheel (V2) – Main is Mini PC:** Alternate drive configuration using an Omni wheel base, with additional feedback from laser sensors (X, Y reference from the wall), sharing CAN BUS communication with the Mecanum board.

2.2 MR2 Buffalo Robot System Architecture

The Buffalo Robot (MR2) is fully autonomous and follows the same two-branch structure: **Mechanic Sim Control** for ball handling mechanisms, and **Base Control** for autonomous navigation.

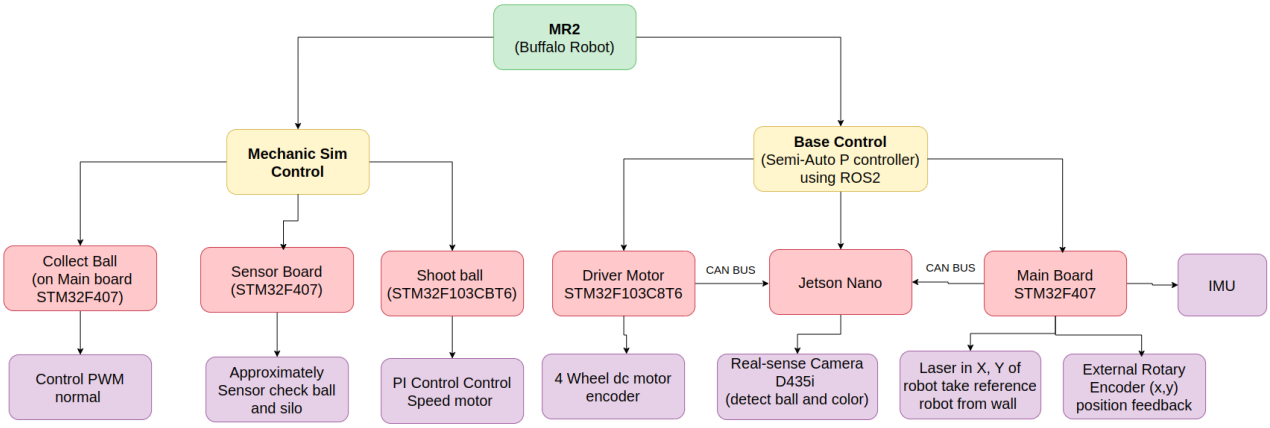


Figure 3: System architecture of MR2 (Buffalo Robot)

Mechanic Sim Control

- **Collect Ball (on Main Board STM32F407):** Controls the PWM signal for the ball-collecting mechanism.
- **Sensor Board (STM32F407):** Uses approximate sensors to check for ball presence and to detect the silo position.
- **Shoot Ball (STM32F103C8T6):** Uses PI control to regulate the motor speed of the shooting mechanism.

Base Control (Semi-Auto P Controller using ROS2)

- **Driver Motor (STM32F103C8T6):** Controls the 4-wheel DC motors with encoder feedback, communicating with the Jetson Nano via CAN BUS.

- **Jetson Nano:** Acts as the onboard computer running ROS2, interfacing with the Intel RealSense D435i camera for ball detection and color classification.
- **Main Board (STM32F407):** Communicates with the Jetson Nano via CAN BUS, interfaces with the IMU for heading (yaw) measurement, laser sensors for X, Y wall-reference positioning, and an external rotary encoder for X, Y position feedback.

3 Lecture Review

3.1 Mecanum Wheel

The mecanum wheel is a form of tireless wheel, with a series of rubberized external rollers obliquely attached to the whole circumference of its rim. These rollers typically each have an axis of rotation at 45° to the wheel plane and at 45° to the axle line.

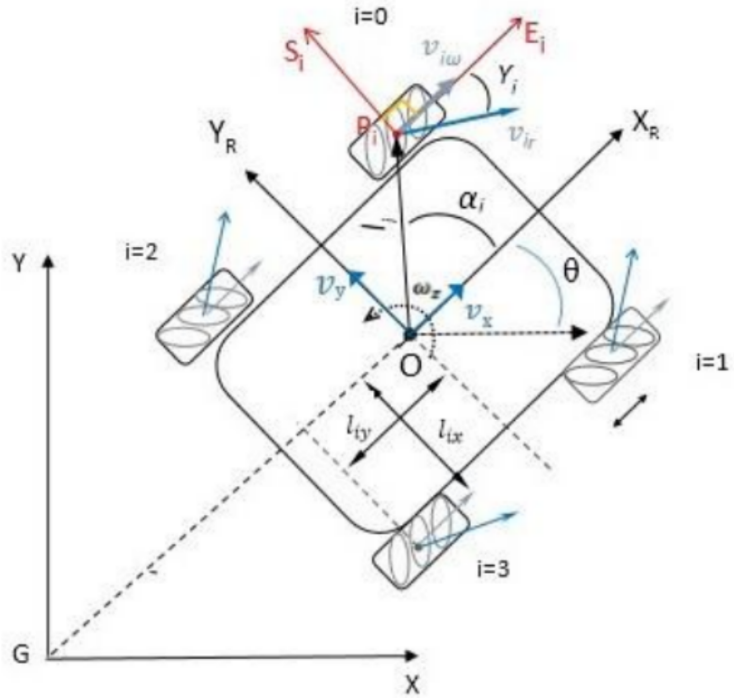


Figure 4: Movement and Dericton of mecanun wheel

The configuration parameter and system velocities are defind as follows:

- $\boldsymbol{x, y, \theta}$: Robot's position (x,y) and its orientation angle θ (The angle between X and X_R).
- **X,G,Y**: Inertial frame; x,y are the coordinates of the reference point O in the
- $\mathbf{X_R O Y_R}$: Robot's base frame; Cartesian coordinate system associated with the movement of the body center.
- $\mathbf{S_i P_i E_i}$: Coordinate system of ith wheel in the wheel's center point P_i.
- $\mathbf{O, Pi}$: The inertial basis of the Robot in Robot's frame and P i = X P i , Y P i the center of the rotation axis of the wheel i.
- $\mathbf{O\tilde{P}_i}$:is a vector that indicates the distance between Robot's center and the center of the wheel ith.

- $\mathbf{l_{ix}}, \mathbf{l_{iy}}$: l_{ix} :half of the distance between front wheels and l_{iy} : half of the distance between front wheel and the base (center of the robot O).
- $\mathbf{l_i}$: distance between wheels and the base (center of the robot O).
- $\mathbf{r_i}$: denotes the radius of the wheel i (Distance of the wheel's center to the roller
- $\mathbf{r_r}$: denotes the radius of the rollers on the wheel.
- $\boldsymbol{\alpha_i}$: the angel between OP_i and X_R .
- $\boldsymbol{\beta_i}$: the angle between S_i and X_R.
- $\boldsymbol{\gamma_i}$: the angle between v_{ir} and E_i.
- $\boldsymbol{\omega}[\boldsymbol{rad/s}]$: wheels angular velocity;
- $\mathbf{V_{i\theta}}[\mathbf{rad/s}]$: wheels angular velocity.
- $\mathbf{V_{ir}}$: the velocity of the passive roller in the wheel i
- $[\boldsymbol{W_s i W_E i \theta_i}]^{\boldsymbol{T}}$: Genralizd velocity of point P_i in the frame $S_i P_i E_i$
- $[\boldsymbol{V_s i V_E i \theta_i}]^{\boldsymbol{T}}$: Generalized velocity of point P_i in the frame $X_R O Y_R$.
- $\mathbf{v_x v_y}[\mathbf{m/s}]$: Robot linear velocity;
- $\boldsymbol{\theta(rad/s)}$: Robot angular velocity; center)

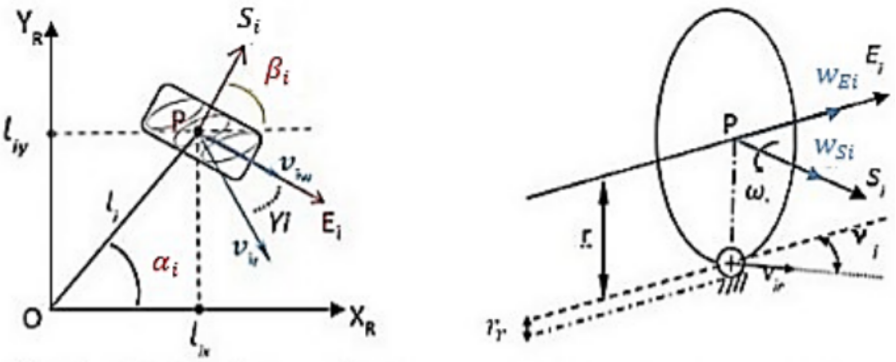


Figure 5: Coordination and motion mecanum wheel

_ **Wheel i and the tangential velocity of the free roller attaced to the wheel touching the floor:**

$$v_{ir} = \frac{1}{cos(45)} r_r \omega_i, \quad W_{Ei} = r_r \omega_i \quad (eq.1)$$

_ **The velocity of the wheel i in the frame $S_i P_i E_i$, can be derived by:**

$$\begin{bmatrix} v_{si} \\ v_{Ei} \end{bmatrix} = \begin{bmatrix} 0 & sin(\gamma_i) \\ r_i & cos(\gamma_i) \end{bmatrix} \begin{bmatrix} \omega_i \\ v_{ir} \end{bmatrix} = T_{Pi}^{W_i} \begin{bmatrix} \omega_i \\ v_{ir} \end{bmatrix} \qquad \text{(eq.2)}$$

_ **The velocity of the wheel's center translated to the $X_R O Y_R$ coordinate system can be achieved by equation 2**:

i	**Wheels** $\boldsymbol{\alpha_i}$	$\boldsymbol{\beta_i}$	$\boldsymbol{\gamma_i}$	l_i	l_{ix}	l_{iy}
0	1sw	$\frac{\pi}{4}$	$\frac{\pi}{2}$	$-\frac{\pi}{4}$	l	lx	ly
1	2sw	$-\frac{\pi}{4}$	$-\frac{\pi}{2}$	$\frac{\pi}{4}$	l	lx	ly
2	3sw	$\frac{3\pi}{4}$	$\frac{\pi}{2}$	$\frac{\pi}{4}$	l	lx	ly
3	4sw	$-\frac{3\pi}{4}$	$-\frac{\pi}{2}$	$-\frac{\pi}{4}$	l	lx	ly

Table 1: Table Parameter of Mecanum Wheel

$$\begin{bmatrix} V_{iX_R} \\ V_{iY_R} \end{bmatrix} = \begin{bmatrix} cos(\beta_i) & -sin(\beta_i) \\ sin(\beta_i) & cos(\beta_i) \end{bmatrix} \begin{bmatrix} v_{si} \\ v_{Ei} \end{bmatrix} = T_{P_i}^{W} T_{R}^{P_i} \begin{bmatrix} \omega_i \\ v_{ir} \end{bmatrix} \quad (eq.3)$$

_ **The robot's motion is planar,we also have:**

$$\begin{bmatrix} v_{iX_R} \\ v_{iY_R} \end{bmatrix} = \begin{bmatrix} 1 & 0 & -l_{iy} \\ 0 & 1 & l_{ix} \end{bmatrix} \begin{bmatrix} V_{X_R} \\ V_{Y_R} \\ \theta_R \end{bmatrix} = T' \begin{bmatrix} V_{X_R} \\ V_{Y_R} \\ \theta_R \end{bmatrix} \quad (eq.4)$$

_ **The inverse kinematic model can be obtained:**

$$T_{P_i}^{W} T_{R}^{P_i} \begin{bmatrix} \omega_i \\ v_{ir} \end{bmatrix} = T' \begin{bmatrix} V_{X_R} \\ V_{Y_R} \\ \theta_R \end{bmatrix} \quad (eq.5)$$

_ **The robot's base velocity (at point O) related to the rotational velocity of the ith wheel can be obtained from eq.5**

$$\begin{bmatrix} \omega_i \\ v_{ir} \end{bmatrix} = (T_{P_i}^{W})^{-1} (T_{R}^{P_i})^{-1} T' \begin{bmatrix} V_{X_R} \\ V_{Y_R} \\ \theta_R \end{bmatrix} \quad (eq.5)$$

Let : $T = (T_{P_i}^{W})^{-1} (T_{R}^{P_i})^{-1} T'$

$$T = \begin{bmatrix} cos(\beta_i) & -sin(\beta_i) \\ sin(\beta_i) & cos(\beta_i) \end{bmatrix}^{-1} \begin{bmatrix} 0 & sin(\gamma_i) \\ r_i & cos(\gamma_i) \end{bmatrix}^{-1} \begin{bmatrix} 1 & 0 & -l_{iy} \\ 0 & 1 & l_{ix} \end{bmatrix}$$

Where: $l_{ix} = l_i cos(\alpha_i) and l_{iy} = l_i sin(\alpha_i)$

$$=> T = -\frac{1}{r} \begin{bmatrix} \frac{cos(\beta_i - \gamma_i)}{sin(\gamma_i)} & \frac{sin(\beta_i - \gamma_i)}{sin(\gamma_i)} & \frac{l_i sin(-\alpha_i + \beta_i - \gamma_i)}{sin(\gamma_i)} \\ -\frac{r cos(\beta_i - \gamma_i)}{sin(\gamma_i)} & -\frac{r sin(\beta_i - \gamma_i)}{sin(\gamma_i)} & -\frac{r l_i sin(-\alpha_i + \beta_i - \gamma_i)}{sin(\gamma_i)} \end{bmatrix}$$

Since there is a relation between independent variables v_{ir} and ω i in each joint and the systems angular and linear velocity, assuming that there is no wheel slipping on the ground, the system inverse kinematic can be obtained by eq.7

_ **How to calculate velocity 4 wheel:**

$$\begin{bmatrix} \omega_1 \\ \omega_2 \\ \omega_3 \\ \omega_4 \end{bmatrix} = -\frac{1}{r} \begin{bmatrix} \frac{cos(\beta_1 - \gamma_1)}{sin(\gamma_1)} & \frac{sin(\beta_1 - \gamma_1)}{sin(\gamma_1)} & \frac{l_1 sin(-\alpha_1 + \beta_1 - \gamma_1)}{sin(\gamma_1)} \\ \frac{cos(\beta_2 - \gamma_2)}{sin(\gamma_2)} & \frac{sin(\beta_2 - \gamma_2)}{sin(\gamma_2)} & \frac{l_1 sin(-\alpha_2 + \beta_2 - \gamma_2)}{sin(\gamma_2)} \\ \frac{cos(\beta_3 - \gamma_3)}{sin(\gamma_3)} & \frac{sin(\beta_3 - \gamma_3)}{sin(\gamma_3)} & \frac{l_1 sin(-\alpha_3 + \beta_3 - \gamma_3)}{sin(\gamma_3)} \\ \frac{cos(\beta_4 - \gamma_4)}{sin(\gamma_4)} & \frac{sin(\beta_4 - \gamma_4)}{sin(\gamma_4)} & \frac{l_1 sin(-\alpha_4 + \beta_4 - \gamma_4)}{sin(\gamma_4)} \end{bmatrix} \begin{bmatrix} Vx \\ Vy \\ \omega_z \end{bmatrix}$$

_ **Parameter $\boldsymbol{\alpha, \beta, \gamma}$ in Mecanum Omnidirection Wheel**.

- We get Inverse Kinematics form:

$$\begin{bmatrix}\omega_1\\ \omega_2\\ \omega_3\\ \omega_4\end{bmatrix} = \frac{1}{r}\begin{bmatrix}1 & -1 & -(lx+ly)\\ 1 & 1 & (lx+ly)\\ 1 & 1 & -(lx+ly)\\ 1 & -1 & (lx+ly)\end{bmatrix}\begin{bmatrix}Vx\\ Vy\\ \omega_z\end{bmatrix}$$

- Forward Kinematics form :

$$\begin{bmatrix}Vx\\ Vy\\ \omega_z\end{bmatrix} = \frac{r}{4}\begin{bmatrix}1 & 1 & 1 & 1\\ -1 & 1 & 1 & -1\\ -\frac{1}{lx+ly} & \frac{1}{lx+ly} & -\frac{1}{lx+ly} & \frac{1}{lx+ly}\end{bmatrix}\begin{bmatrix}\omega_1\\ \omega_2\\ \omega_3\\ \omega_4\end{bmatrix}$$

3.2 Omni Wheel

Omni wheels or poly wheels, similar to Mecanum wheels, are wheels with small discs (called rollers) around the circumference which are perpendicular to the turning direction. The effect is that the wheel can be driven with full force, but will also slide laterally with great ease.

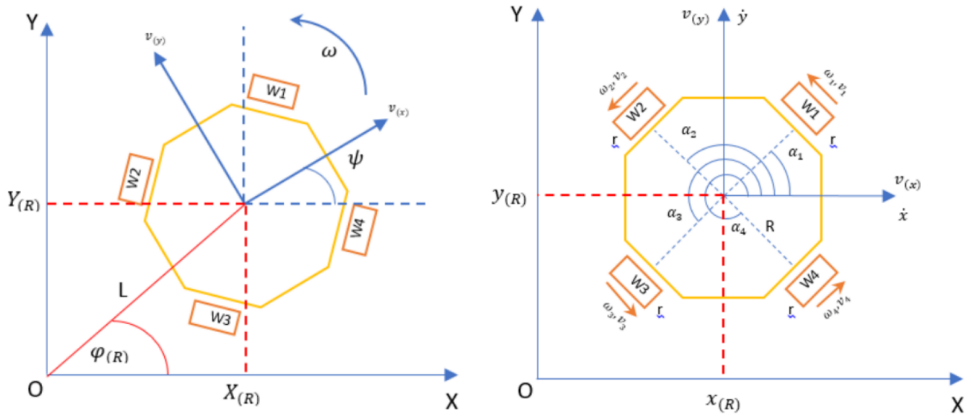


Figure 6: Movement and Direction of OMNI Wheel

The configuration parameter:

- $\boldsymbol{\alpha_1, \alpha_2, \alpha_3, \alpha_4}$ = The angle formed between the wheels and the robot reference point.
- $\boldsymbol{\gamma}$ = The angular from roller to wheel.
- $\boldsymbol{\psi_R}$ = is the direction of movement of the robot at global coordinates.
- $\boldsymbol{\omega_W}$ = The angular velocity of each wheel.
- $\mathbf{V_W}$ = The linear velocity of each Wheel.
- **XOY** = global coordinates are notified using $V(g) = [\dot{X}, \dot{Y}, \omega]$.
- **R** = is the distance from the Wheel to the center of the robot.
- **r** = is the radius of the omni wheel.
- $\mathbf{V_R} = [\mathbf{V_x}, \mathbf{V_y}]$ = The position of the robot's coordinate.
- $\boldsymbol{X_{(g)} = [X_R Y_R \psi]^T}$ = Orientation to global coordinates.
- $\boldsymbol{\omega}$ = is the notation of the angular velocity of the robot to the global coordinates.

- $\boldsymbol{\Phi_n}$ = is a notation of the direction of movement of the robot to global coordinates.
- $\mathbf{L}$ = is the notation of the resultant linear velocity of the robot.

_ **The OMNI Wheel equation**

$$\boldsymbol{\omega_i = h_i(\psi)\dot{q}} = \begin{bmatrix} \frac{1}{r_i} & \frac{tan\gamma_i}{r_i} \end{bmatrix} \begin{bmatrix} \boldsymbol{cos\alpha_i} & \boldsymbol{sin\alpha_i} \\ \boldsymbol{-sin\alpha_i} & \boldsymbol{cos\alpha_i} \end{bmatrix} \begin{bmatrix} \boldsymbol{-y_i} & \mathbf{1} & \mathbf{0} \\ \boldsymbol{x_i} & \mathbf{0} & \mathbf{1} \end{bmatrix} \begin{bmatrix} \mathbf{1} & \mathbf{0} & \mathbf{0} \\ \mathbf{0} & \boldsymbol{cos\psi} & \boldsymbol{sin\psi} \\ \mathbf{0} & \boldsymbol{-sin\psi} & \boldsymbol{cos\psi} \end{bmatrix} \begin{bmatrix} \dot{\psi} \\ \dot{x} \\ \dot{y} \end{bmatrix}$$

$$\boldsymbol{h_i(\phi)} = \frac{\mathbf{1}}{\boldsymbol{r_i cos\gamma_i}} \begin{bmatrix} \boldsymbol{x_i sin(\alpha_i + \gamma_i) - y_i cos(\beta_i + \gamma_i)} \\ \boldsymbol{cos(\alpha_i + \gamma_i + \psi)} \\ \boldsymbol{sin(\alpha_i + \gamma_i + \psi)} \end{bmatrix}^{\boldsymbol{T}}$$

Where: $\gamma = 0$ because in omni wheel is roller parallel with Wheel.

$$\boldsymbol{\omega_i} = \frac{\mathbf{1}}{\boldsymbol{r}}(\boldsymbol{-sin(\psi + \alpha_i)\dot{x} + cos(\psi + \alpha_i)\dot{y} + R\dot{\psi}})$$

- **4 Wheel**

$$\begin{bmatrix} \boldsymbol{\omega_1} \\ \boldsymbol{\omega_2} \\ \boldsymbol{\omega_3} \\ \boldsymbol{\omega_4} \end{bmatrix} = \frac{1}{r} \begin{bmatrix} \boldsymbol{-sin(\psi + \alpha_1)} & \boldsymbol{cos(\psi + \alpha_1)} & \boldsymbol{R} \\ \boldsymbol{-sin(\psi + \alpha_2)} & \boldsymbol{cos(\psi + \alpha_2)} & \boldsymbol{R} \\ \boldsymbol{-sin(\psi + \alpha_3)} & \boldsymbol{cos(\psi + \alpha_3)} & \boldsymbol{R} \\ \boldsymbol{-sin(\psi + \alpha_4)} & \boldsymbol{cos(\psi + \alpha_4)} & \boldsymbol{R} \end{bmatrix} \begin{bmatrix} \dot{x} \\ \dot{y} \\ \dot{\psi} \end{bmatrix}$$

Where: $\alpha_1 = \frac{\pi}{4}, \alpha_2 = \frac{3\pi}{4}, \alpha_3 = \frac{5\pi}{4}, \alpha_4 = \frac{7\pi}{4}$, and take $\psi = 0$ because robot start at angle 0 degree.

- **Inverse kinematic Form:**

$$\begin{bmatrix} \boldsymbol{\omega_1} \\ \boldsymbol{\omega_2} \\ \boldsymbol{\omega_3} \\ \boldsymbol{\omega_4} \end{bmatrix} = \frac{1}{r} \begin{bmatrix} \boldsymbol{-sin(\frac{\pi}{4})} & \boldsymbol{cos(\frac{\pi}{4})} & \boldsymbol{R} \\ \boldsymbol{-sin(\frac{\pi}{34})} & \boldsymbol{cos(\frac{3\pi}{4})} & \boldsymbol{R} \\ \boldsymbol{-sin(\frac{5\pi}{4})} & \boldsymbol{cos(\frac{5\pi}{4})} & \boldsymbol{R} \\ \boldsymbol{-sin(\frac{7\pi}{4})} & \boldsymbol{cos(\frac{7\pi}{4})} & \boldsymbol{R} \end{bmatrix} \begin{bmatrix} \dot{x} \\ \dot{y} \\ \dot{\psi} \end{bmatrix}$$

- **Forward Kinematic Form:**

$$\begin{bmatrix} \dot{x} \\ \dot{y} \\ \dot{\psi} \end{bmatrix} = \frac{r}{4} \begin{bmatrix} \boldsymbol{-sin(\frac{\pi}{4})} & \boldsymbol{-sin(\frac{3\pi}{4})} & \boldsymbol{-sin(\frac{5\pi}{4})} & \boldsymbol{-sin(\frac{7\pi}{4})} \\ \boldsymbol{cos(\frac{\pi}{4})} & \boldsymbol{cos(\frac{3\pi}{4})} & \boldsymbol{cos(5\frac{\pi}{4})} & \boldsymbol{cos(7\frac{\pi}{4})} \\ \boldsymbol{\frac{1}{R}} & \boldsymbol{\frac{1}{R}} & \boldsymbol{\frac{1}{R}} & \boldsymbol{\frac{1}{R}} \end{bmatrix} \begin{bmatrix} \boldsymbol{\omega_1} \\ \boldsymbol{\omega_2} \\ \boldsymbol{\omega_3} \\ \boldsymbol{\omega_4} \end{bmatrix}$$

3.3 PID CONTROLLER

A PID-Controller is a widely used feedback control mechanism in engineering and industrial appli- cations. It is employed to regulate processes and systems by continuously adjusting an output based on the difference between a desired setpoint and the measured process variable.

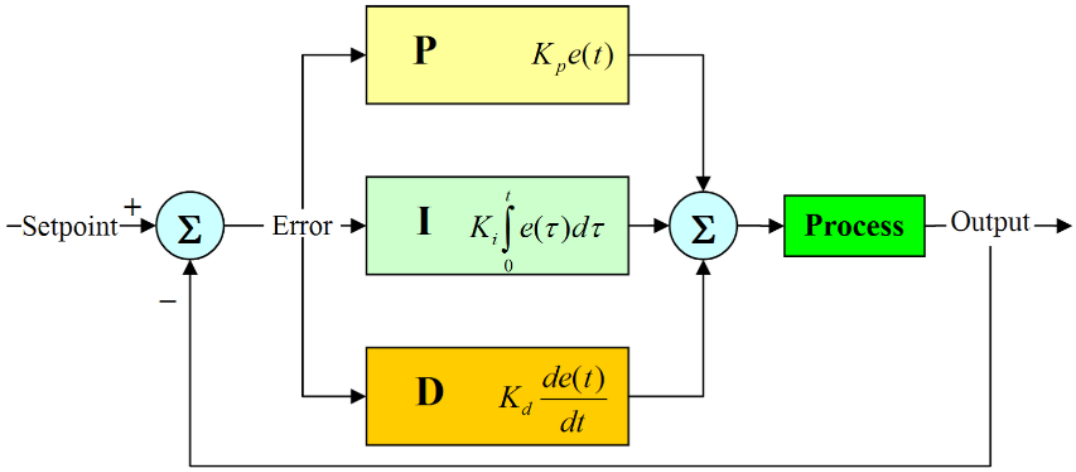


Figure 7: PID Controller Block diagram

3.3.1 PID Algorithm

- **Step1: Calculate Error**
  Error = Reference - Measurement.
- **Step2:Calculate P,I,D**
  Proportional = Kp*Error_cur
  Integral = Integral_prev + Ki*(Error_cur + Error_prev)*sampling_time
  Derivative = Kd*(Error_cur - Error_prev)/sampling_time
  where kp,ki,kd (you can tuning on Modelling thought MATLAB Simulink or tuning by hand(observe)
- **Step3:Calculate Output**
  Output = Proportional + Integral + Derivative.

3.3.2 Integral Windup

Integrator windup or reset windup, refers to the situation in a PID controller where a large change in setpoint occurs (say a positive change) and the integral term accumulates a significant error during the rise (windup), thus overshooting and continuing to increase as this accumulated error is unwound (offset by errors in the other direction). The specific problem is the excess overshooting.
How to use Integral windup:

- Set Integral Min, Integral Max.
- When you use Integral winup:
  $if(Integral >= IntegralMax)$
  $\{Integral = IntegralMax;$

  $\}$
  $elseif(Integral <= IntegralMin)$
  $\{Integral = IntegralMin;$

  $\}$

$else\{Integral = Integral;$

}

3.3.3 Low pass filter

A low-pass filter in a PID (Proportional-Integral-Derivative) controller is a component that filters out high-frequency noise from the derivative (D) term of the PID controller's output. The purpose of the low-pass filter is to smooth the derivative term and reduce its sensitivity to rapid, high-frequency changes in the error signal. By doing so, the controller can better distinguish between genuine changes in the process variable and short-term noise, resulting in more stable and accurate control.
How to use Low pass filter in PID:

- Derivative of Positional Error after low pass filter:

$$\boldsymbol{E(t) = (1 - \alpha) * E(t - 1) + \alpha * \frac{dError(t)}{dt}}$$

- α Smoothing Factor (smaller for greater smoothing) $(0 < \alpha < 1)$

3.3.4 Output Saturation

This objective is achieved by including a saturation block that ensure the output is always in the range $[OUT_{min}, OUT_{max}]$
How to use Output saturation:

- Output = Propotional + Integral + Derivative.
  $if(Output >= Outputmax)\{$
  . $Output = Outputmax;$
  }
  $elseif(Output <= Outputmin)\{$
  . $Output = Outputmin;$
  }
  $else\{$
  . $Output = Output;$
  }

3.4 LQR Controller

LQR is a method that calculates the optimal feedback gain K. The feedback gain can determined by tuning of Q and R. The feedback gain is used to control the system in control signal form.The main focus of an LQR control system is to achieve and hold a target configuration for a given linear system using an optimal control law.

- An LQR control system generates the control law using four matrices:
  – A Matrix:physical dynamics
  – B Matrix:control dynamics
  – Q Matrix:state cost
  – R Matrix:control cost

- LQR's goal is to minimize the "cost" of the state error, while minimizing the "cost" of actuator effort.

- LQR determines the point along the cost curve where the "cost function" is minimized. . . thereby balancing both objectives of keeping state error and actuator effort minimized.

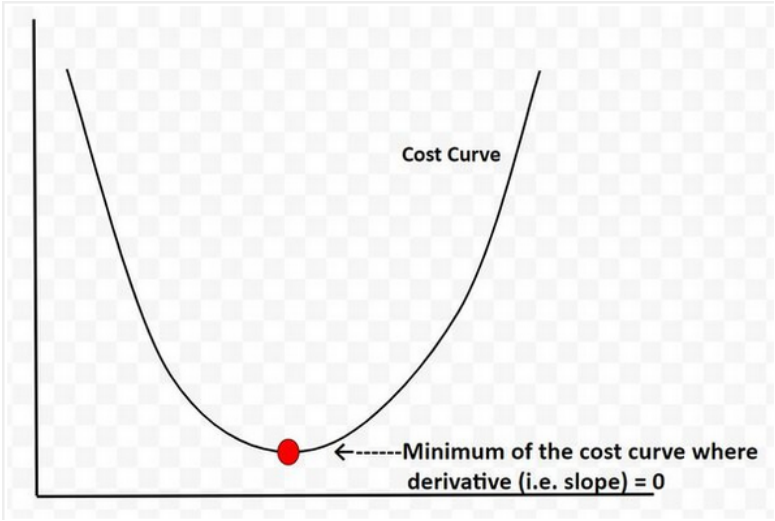


Figure 8: Cost curve of LQR

3.4.1 LQR Algorithm

- **Setting up the optimization problem**
  - **Positive Semidefinite**

$$\mathbf{X^T Q X >= 0}$$

  - **Positive definite**

$$\mathbf{U^T R U > 0}$$

  - If Q "is bigger" than R => fast regulation of X− >0,U is Large
  - If R "is bigger" than Q => slow regulation of X− >0,U is Small.

- **Solving the optimization problem**
  We are using one of the optimal control method which is Linear Quadratic Regulator. Minimize the cost:

$$\mathbf{J = \frac{1}{2} \sum_{k=0}^{N} (X_{k+1}^T Q X_{k+1} + U_k^T R U_k)} \quad \text{(Discrete-time)}$$

  OR

$$\mathbf{J = \int_0^\infty (X^T Q X + U^T R U) dt} \quad \text{(Continuous-time)}$$

- **Mathematics Model**

$$\mathbf{X_t = A_{t-1} X_{t-1} + B_{t-1} U_{t-1}}$$

  The General form of the linearized model Differential Wheel:

$$\mathbf{X_t} = \begin{bmatrix} 1 & 0 & 0 \\ 0 & 1 & 0 \\ 0 & 0 & 1 \end{bmatrix} \begin{bmatrix} x_{t-1} \\ y_{t-1} \\ \theta_{t-1} \end{bmatrix} + \begin{bmatrix} cos(\theta_{t-1})dt & 0 \\ sin(\theta_{t-1})dt & 0 \\ 0 & dt \end{bmatrix} \begin{bmatrix} v_{t-1} \\ \omega_{t-1} \end{bmatrix}$$

- **Finite-horizon discrete-time**

$$\mathbf{J = X_{H_p}^T Q_{H_p} X_{H_p} + \sum_{k=0}^{H_p-1} (X_k^T Q X_k + U_k^T R U_k + 2 X_k^T N U_k)}$$

Where: H_p: is the time horizon, N: is number step horizon, k: is time current.
from terminal condition $P_{H_p} = Q_{H_p}$.
Note that U_{H_p} is not defined, since x is driven to its final state X_{H_p} by $AX_{H_p-1} + BU_{H_p-1}$.

  - P_k is found iteratively backwards in time by the dynamic Riccati equation:

$$\mathbf{P_k = A^T P_k A - (A^T P_k B + N)(R + B^T P_k B)^{-1}(B^T P_k A + N^T) + Q}$$

  - Optimal feedback gain:

$$\mathbf{F_k = (R + B^T P_{k+1} B)^{-1}(B^T P_{k+1} A + N^T)}$$

  - Optimal control sequence minimizing the performance index is given by:

$$\mathbf{u_k = -F_k X_k}$$

- **Infinite-horizon, discrete-time**

$$\mathbf{J = \sum_{k=0}^{\infty} (X_k^T Q X_k + U_k^T R U_k + 2 X_k^T N U_k)}$$

  - P is the unique positive definite solution to the discrete time algebraic Riccati equation (DARE):

$$\mathbf{P = A^T P A - (A^T P B + N)(R + B^T P B)^{-1}(B^T P A + N^T) + Q}$$

  - Optimal feedback gain:

$$\mathbf{F = (R + B^T P B)^{-1}(B^T P A + N^T)}$$

  - Optimal control sequence minimizing the performance index is given by:

$$\mathbf{u_k = -F X_k}$$

- **Note When you use to write in program is using (Finite-horizon discrete-time).**

3.5 NONLINEAR MODEL PREDICTIVE CONTROL (NMPC) CONTROLLER

Model Predictive Control is one of the advanced technique control process that can obtained an optimal solution over a period with finite and infinite horizon. Model Predictive Control(MPC) or Receding Horizontal Control (RHC) is also an original framework of feedback control of the nonlinear dynamic system. Nonlinear Model Predicted Control(NMPC) is a variant of model predictive control that is characterized by the use of nonlinear system models in the prediction.

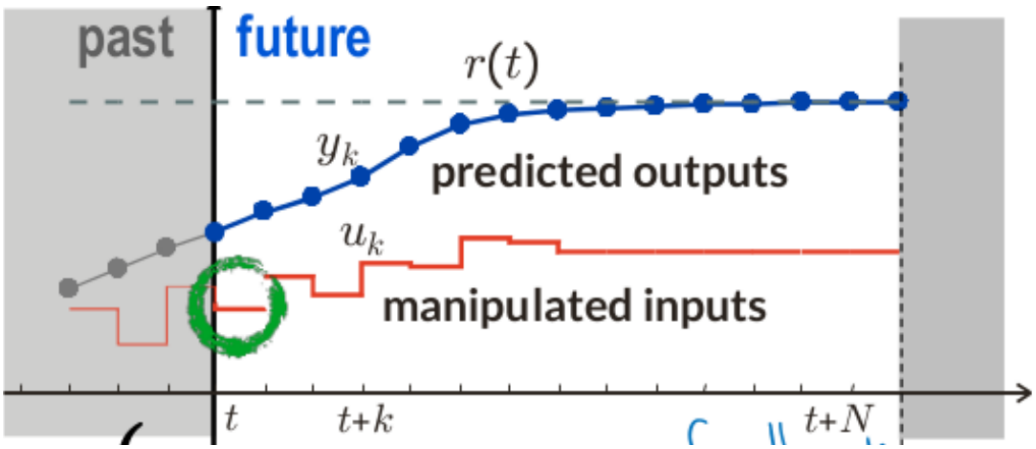


Figure 9: MPC Prdicted output

3.6 NMPC Algorithm Discrete-time

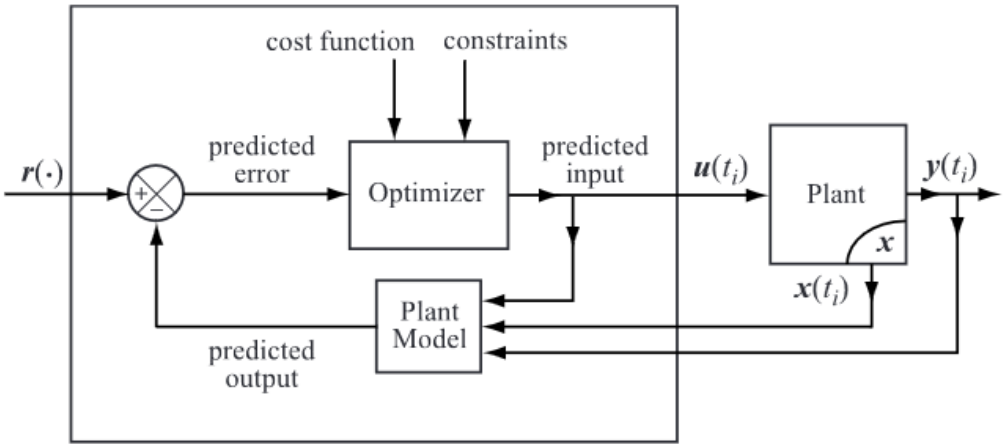


Figure 10: Block diagram of Model Predicted Controller

- **Linear prediction model:**
  - State Space model: $\mathbf{X_{k+1} = AX_k + BU_k}$ (eq.1)
  - Observation model: $\mathbf{Y_k = CX_k}$ (eq.2)
- **The optimal control problem (quadratic performance index):**
  - Backward difference State : $\mathbf{\Delta X_{k+1} = X_{k+1} - X_k}$
  - Backward difference Input : $\mathbf{\Delta U_{k+1} = U_{k+1} - U_k}$
    And given from (eq.1): $\mathbf{\Delta X_{k+1} = A\Delta X_k + B\Delta U_k}$ (eq.3)
  - Backward difference Output : $\mathbf{\Delta Y_k = Y_k - Y_{k-1}}$
    And given from (eq.2): $\mathbf{\Delta Y_{k+1} = C\Delta X_{k+1} = CA\Delta X_k + B\Delta U_k}$
    $=> Y_{k+1} = Y_k + CA\Delta X_k + B\Delta U_k$ (eq.4)
    We Combine Equation(3) and (4):

$$\begin{bmatrix} \boldsymbol{\Delta X_{k+1}} \\ \boldsymbol{Y_{k+1}} \end{bmatrix} = \begin{bmatrix} \boldsymbol{A} & \boldsymbol{0} \\ \boldsymbol{CA} & \boldsymbol{I_p} \end{bmatrix} \begin{bmatrix} \boldsymbol{\Delta X_{k+1}} \\ \boldsymbol{Y_k} \end{bmatrix} \begin{bmatrix} \boldsymbol{B} \\ \boldsymbol{CB} \end{bmatrix} \boldsymbol{\Delta U_k}$$

We now define the augmented state vector : $\mathbf{Xa_k} = \begin{bmatrix} \Delta X_k \\ Y_k \end{bmatrix}$

Let: $\boldsymbol{Y_k} = \begin{bmatrix} 0 & \boldsymbol{I_p} \end{bmatrix} \begin{bmatrix} \boldsymbol{\Delta X_k} \\ \boldsymbol{Y_k} \end{bmatrix}, \boldsymbol{A_a} = \begin{bmatrix} \boldsymbol{A} & \boldsymbol{0} \\ \boldsymbol{CA} & \boldsymbol{I_p} \end{bmatrix}, \boldsymbol{B_a} = \begin{bmatrix} \boldsymbol{\Delta X_{k+1}} \\ \boldsymbol{Y_k} \end{bmatrix}, \boldsymbol{C_a} = \begin{bmatrix} 0 & \boldsymbol{I_p} \end{bmatrix}$

- **Update Model predicted**:

$$\mathbf{Xa_{k+1}} = \mathbf{A_a Xa_k} + \mathbf{B_a \Delta U_k} \quad (eq.5)$$

$$\mathbf{Y_k} = \mathbf{C_a Xa_k} \quad (eq.6)$$

- **State error:** $\mathbf{E_k} = \mathbf{r_{p_k}} - \mathbf{Y_k}$

$$\min_{\mathbf{z}} \mathbf{E_N^T P E_N} \sum_{\mathbf{k=0}}^{\mathbf{N-1}} \mathbf{E_k^T Q E_k} + \mathbf{\Delta U_k^T R \Delta U_k}$$

Subject to :

$$\mathbf{X_{min}} \leq \mathbf{X_k} \leq \mathbf{X_{max}}, \quad \mathbf{k} = [\mathbf{0}, \mathbf{N}]$$

$$\mathbf{U_{min}} \leq \mathbf{U_k} \leq \mathbf{U_{max}}, \quad \mathbf{k} = [\mathbf{0}, \mathbf{N-1}]$$

Where: $R = R^T > 0; Q = Q^T \geq 0; P = P^T \geq 0; z = \begin{bmatrix} u_0 & u_1 & \dots & u_{N-1} \end{bmatrix}$

- $\mathbf{r_p}$ = the reference state of the system or desired input state.
- $\mathbf{N}$ = number prediction horizon.

- **For update state has two methode:**

$$\dot{\mathbf{X}} = \mathbf{f}(\mathbf{X}, \mathbf{U(t)})$$

Example model of mecanum wheel:

$$\begin{bmatrix} \boldsymbol{V_x} \\ \boldsymbol{V_y} \\ \boldsymbol{\omega_z} \end{bmatrix} = \frac{\boldsymbol{r}}{4} \begin{bmatrix} \boldsymbol{cos(\theta)} & \boldsymbol{-sin(\theta)} & \boldsymbol{0} \\ \boldsymbol{sin(\theta)} & \boldsymbol{cos(\theta)} & \boldsymbol{0} \\ \boldsymbol{0} & \boldsymbol{0} & \boldsymbol{1} \end{bmatrix} \begin{bmatrix} \boldsymbol{1} & \boldsymbol{1} & \boldsymbol{1} & \boldsymbol{1} \\ \boldsymbol{-1} & \boldsymbol{1} & \boldsymbol{1} & \boldsymbol{-1} \\ \boldsymbol{-\frac{1}{l_x+l_y}} & \boldsymbol{\frac{1}{l_x+l_y}} & \boldsymbol{-\frac{1}{l_x+l_y}} & \boldsymbol{\frac{1}{l_x+l_y}} \end{bmatrix} \begin{bmatrix} \boldsymbol{U_1} \\ \boldsymbol{U_2} \\ \boldsymbol{U_3} \\ \boldsymbol{U_4} \end{bmatrix}$$

- Euler Discretization:

$$\begin{bmatrix} \boldsymbol{x_{k+1}} \\ \boldsymbol{y_{k+1}} \\ \boldsymbol{\theta_{k+1}} \end{bmatrix} = \begin{bmatrix} \boldsymbol{x_k} \\ \boldsymbol{y_k} \\ \boldsymbol{\theta_k} \end{bmatrix} + \boldsymbol{\Delta T} \begin{bmatrix} \boldsymbol{V_{x_k}} \\ \boldsymbol{V_{y_k}} \\ \boldsymbol{\omega_{z_k}} \end{bmatrix}$$

- Runge-Kutta 4th (RK4):

$$\boldsymbol{k_1} = \boldsymbol{f(X, U_k)}$$

$$\mathbf{k_2} = \mathbf{f}(\mathbf{X_k} + \frac{\mathbf{\Delta T}}{\mathbf{2}}\mathbf{k_1}, \mathbf{U_k})$$

$$\mathbf{k_3} = \mathbf{f}(\mathbf{X_k} + \frac{\mathbf{\Delta T}}{\mathbf{2}}\mathbf{k_2}, \mathbf{U_k})$$

$$\mathbf{k_4} = \mathbf{f}(\mathbf{X_k} + \mathbf{\Delta T k_3}, \mathbf{U_k})$$

$$\mathbf{X_{k+1}} = \mathbf{X_k} + \frac{\mathbf{\Delta T}}{\mathbf{6}}(\mathbf{k_1} + \mathbf{2k_2} + \mathbf{2k_3} + \mathbf{k_4})$$

- **Nonlinear Program Problem(NLP):A standard problem formulation in numerical optimization.** Problem decision variables :

- Single shooting: $\omega = [u_0, u_1, ..., u_{N-1}]$
- Multiple shooting: $\omega = [x_0, x_1, ..., x_N, u_0, u_1, ..., u_{N-1}]$

$$\min_{\omega} \Phi(\boldsymbol{F}(\omega, \boldsymbol{X_N}), \omega)$$

subject to

$$\boldsymbol{g_1}(\boldsymbol{F}(\omega, \boldsymbol{X_N}), \omega) \leq \boldsymbol{0} \qquad \text{Inequality constraints}$$

$$\boldsymbol{g_2}(\boldsymbol{F}(\omega, \boldsymbol{X_N}), \omega) = \begin{bmatrix} \bar{\boldsymbol{X}}_{\boldsymbol{0}} - \boldsymbol{X_0} \\ \boldsymbol{f}(\boldsymbol{X_0}, \boldsymbol{U_0}) - \boldsymbol{X_1} \\ \vdots \\ \boldsymbol{f}(\boldsymbol{X_{N-1}}, \boldsymbol{U_{N-1}}\boldsymbol{0} - \boldsymbol{X_N} \end{bmatrix} = \boldsymbol{0} \qquad \text{Equality constraints}$$

3.7 Extended Kalman Filter(EKF)

The Extended Kalman Filter is an algorithm that leverages our knowledge of the physics of motion of the system (i.e. the state space model) to make small adjustments to (i.e. to filter) the actual sensor measurements (i.e. what the robot's sensors actually observed) to reduce the amount of noise, and as a result, generate a better estimate of the state of a system.

- The **state estimate** for the previous timestep t-1.
- The **time interval** dt from one timestep to the next.
- The **linear and angular velocity** of the car at the previous time step t-1 (i.e. previous control inputs. . . i.e. commands that were sent to the robot to make the wheels rotate accordingly).
- An **estimate of random noise** (typically small values. . . when we send velocity commands to the car, it doesn't move exactly at those velocities so we add in some random noise).

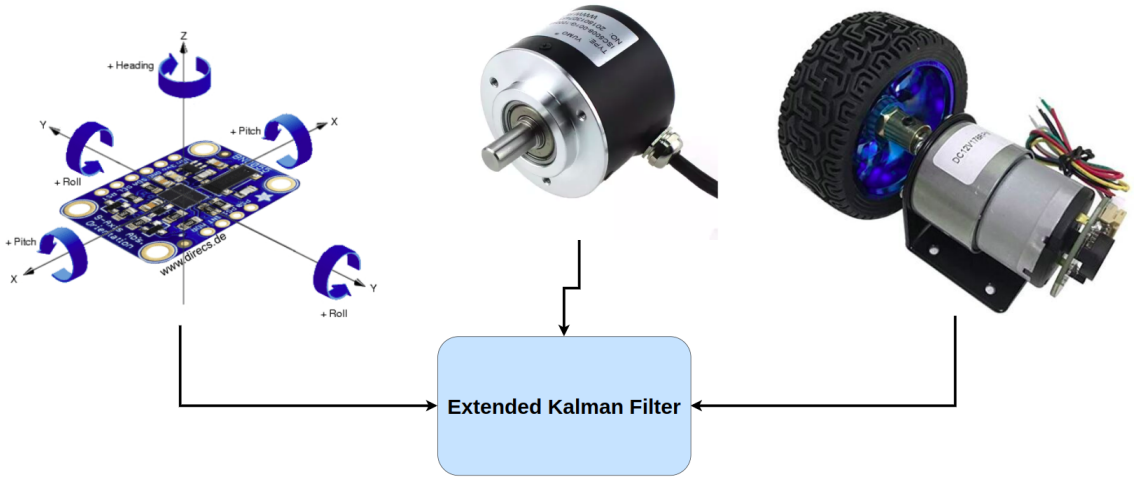


Figure 11: Extended Kalman Filter with Sensor (IMU, External Rotary Encoder, Wheel encoder)

3.7.1 Extended Kalman Filter Algorithm

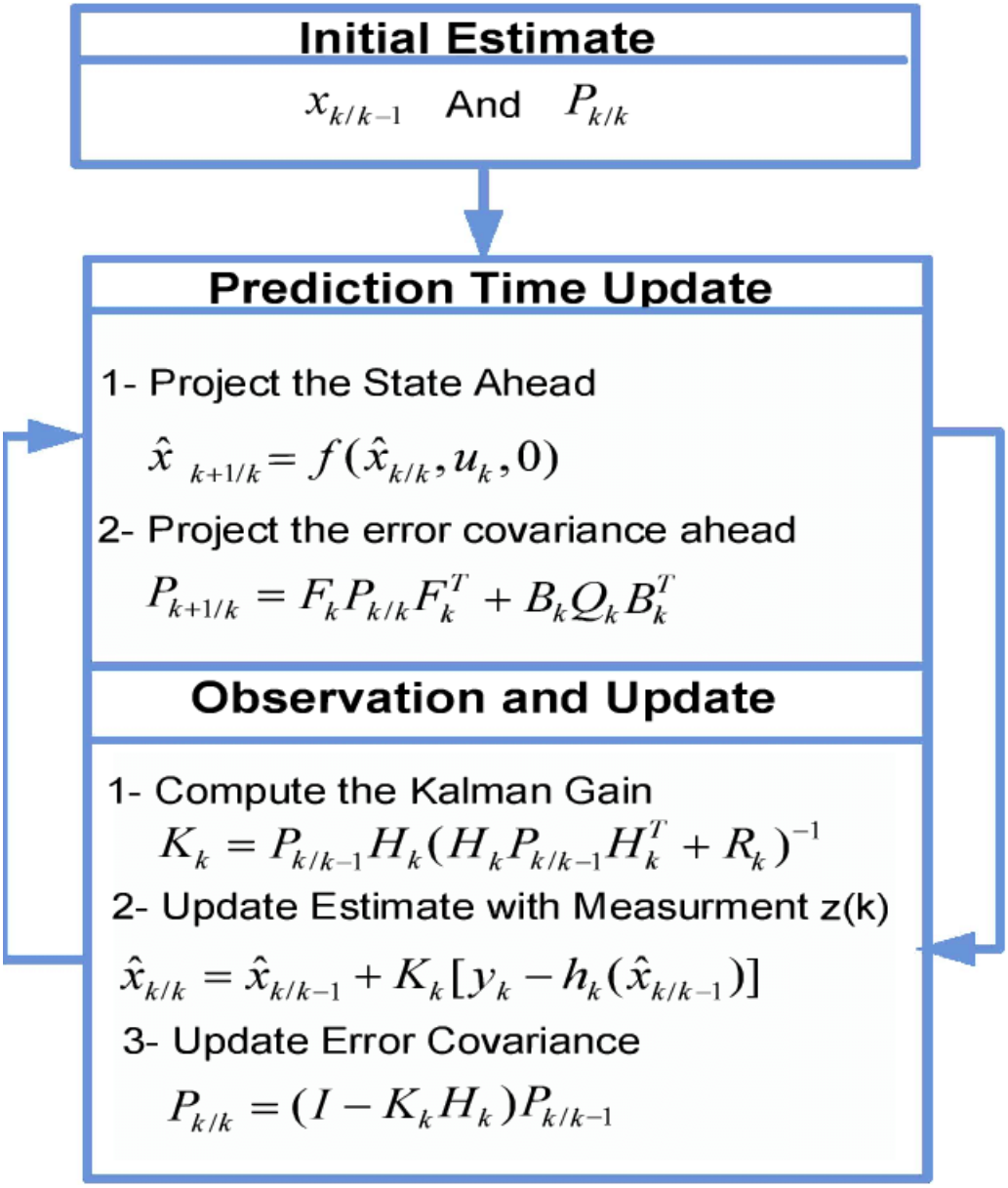


Figure 12: Flow to use Extended Kalman Filter

1. **Initialization**:
   - For the first iteration of EKF, we start at time k. In other words, for the first run of EKF, we assume the current time is k.
   - We initialize the state vector and control vector for the previous time step k-1.
2. **Predicted State Estimate**:

$$\hat{\mathbf{X}}_{\mathbf{k|k-1}} = \mathbf{f}(\hat{\mathbf{X}}_{\mathbf{k-1|k-1}}, \mathbf{u_k}) = \mathbf{A_{k-1}} \mathbf{X_{k-1|k-1}} + \mathbf{B_{k-1}} \mathbf{U_{k-1}} + \mathbf{V_{k-1}}$$

   - We use the state space model, the state estimate for timestep k-1, and the control input vector at the previous time step (e.g. at time k-1) to predict what the state would be for time k (which is the current timestep).

- That equation above is the same thing as our equation below. Remember that we used t in my earlier tutorials.

3. **Predicted Covariance of the State Estimate**

$$\mathbf{P_{k|k-1}} = \mathbf{F_k P_{k-1|k-1} F_k^T} + \mathbf{Q_k}$$

- $P_{k-1|k-1}$ is a square matrix. It has the same number of rows (and columns) as the number of states in the state vector x.
- P (or, commonly, sigma) is a 3×3 matrix. The P matrix has variances on the diagonal and covariances on the off-diagonal.
- we initialize $P_{k-1|k-1}$ to some guessed values (eg 0.1 along the diagonal part of the matrix and 0s elsewhere).

$$\boldsymbol{P_{k-1|k-1}} = \begin{bmatrix} \mathbf{0.1} & \mathbf{0} & \mathbf{0} \\ \mathbf{0} & \mathbf{0.1} & \mathbf{0} \\ \mathbf{0} & \mathbf{0} & \mathbf{0.1} \end{bmatrix}$$

- F_k = are equivalent to A_{k-1} respectively, from my state space model tutorial.
- Q_k = is the state model noise covariance matrix. It is also a 3×3 matrix in our running robot car example because there are three states.

$$\boldsymbol{Q_k} = \begin{bmatrix} \boldsymbol{cov(x,x)} & \boldsymbol{cov(x,y)} & \boldsymbol{cov(x,\gamma)} \\ \boldsymbol{cov(y,x)} & \boldsymbol{cov(y,y)} & \boldsymbol{cov(y,\gamma)} \\ \boldsymbol{cov(\gamma,x)} & \boldsymbol{cov(\gamma,y)} & \boldsymbol{cov(\gamma,\gamma)} \end{bmatrix}$$

- The covariance between two variables that are the same is actually the variance. For example, Cov(x, x) = Var(x).
- We can start by letting Q be the identity matrix and tweak the values through trial and error.

$$\boldsymbol{Q_k} = \begin{bmatrix} \mathbf{1} & \mathbf{0} & \mathbf{0} \\ \mathbf{0} & \mathbf{1} & \mathbf{0} \\ \mathbf{0} & \mathbf{0} & \mathbf{1} \end{bmatrix}$$

- when Q is large, it means we trust our actual sensor observations more than we trust our predicted sensor measurements from the observation model. . . more on this later in this tutorial.

4. **Innovation or Measurement Residual**

$$\mathbf{\tilde{y}_k} = \mathbf{z_k} - \mathbf{h}(\mathbf{\hat{X}_{k|k-1}})$$

- we calculate the difference between actual sensor observations and predicted sensor observations.
- z_k is the observation vector. It is a vector of the actual readings from our sensors at time k.Matrix in z k in mobile robot:

$$\boldsymbol{z_k} = \begin{bmatrix} \boldsymbol{x_k} \\ \boldsymbol{y_k} \\ \boldsymbol{\gamma_k} \end{bmatrix}$$

- $h(\hat{X}_{k|k-1})$ is our observation model. It represents the predicted sensor measurements at time k given the predicted state estimate at time k from Step 2. That hat symbol above x means “predicted” or “estimated”.

$$\boldsymbol{h(\hat{X}_{k|k-1}) = H_k \hat{X}_{k|k-1} + W_k}$$

- Measurement matrix H k (which is used to convert the predicted state estimate at time k into predicted sensor measurements at time k)

$$\boldsymbol{H_k} = \begin{bmatrix} \mathbf{1} & \mathbf{0} & \mathbf{0} \\ \mathbf{0} & \mathbf{1} & \mathbf{0} \\ \mathbf{0} & \mathbf{0} & \mathbf{1} \end{bmatrix}$$

5. **Innovation (or residual) Covariance**

$$\mathbf{S_k = H_k P_{k|k-1} H_k^T + R_k}$$

- use the predicted covariance of the state estimate $P_{k|k-1}$
- The measurement matrix H_k and its transpose.
- R_k (sensor measurement noise covariance matrix. . . which is a covariance matrix that has the same number of rows as sensor measurements and same number of columns as sensor measurements)
- To start out by making R_k the identity through trial and error

$$R_k = \begin{bmatrix} 1 & 0 & 0 \\ 0 & 1 & 0 \\ 0 & 0 & 1 \end{bmatrix}$$

6. **Near-optimal Kalman Gain**

$$\mathbf{K_k = P_{k|k-1} H_k^T S_k^{-1}}$$

- If sensor measurement noise is large, then K approaches 0, and sensor measurements will be mostly ignored.
- If prediction noise (using the dynamical model/physics of the system) is large, then K approaches 1, and sensor measurements will dominate the estimate of the state [x,y,yaw angle].

7. **Updated State Estimate**

$$\mathbf{\hat{x}_{k|k} = \hat{x}_{k|k-1} + K_k \tilde{y}_k}$$

- we calculate an updated (corrected) state estimate

8. **Updated Covariance of the State Estimate**

$$\mathbf{P_{k|k} = (I - K_k H_k) P_{k|k-1}}$$

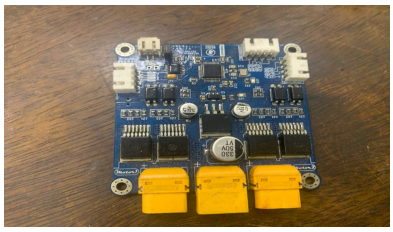
4 Methodology

4.1 Components

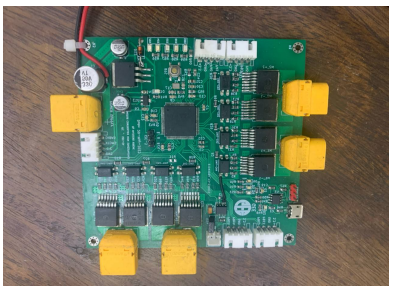
This section describes the hardware components used in the design of MR1 (Farmer Robot) and MR2 (Buffalo Robot). Table 2 summarizes all components, followed by grouped figures showing each component.

Table 2: List of Components

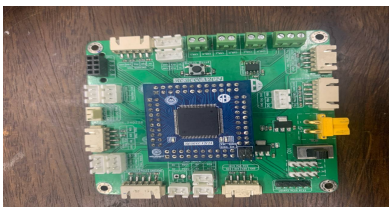
No.	Component	Purpose
1	IMU BNO055 (Black)	Measure heading (yaw angle) for feedback control
2	Laser SICK	Measure X, Y reference distance from wall
3	External Rotary Encoder	Measure (X, Y) position via odometry
4	DC Motor 24V 630 RPM	Drive motor for each wheel (Mecanum/Omni base)
5	DC Motor 555	Drive auxiliary mechanisms (collect/shoot)
6	Stepper Motor NEMA 19 (Gearbox)	Control aiming angle of shooter mechanism
7	Driver 2 Motor	Control 2 DC motors (auxiliary mechanisms)
8	Driver 4 Motor	Control 4 DC motors (drive wheel base)
9	Driver Stepper Motor TB6600	Microstepping control of NEMA 19 stepper motor
10	Main Board (STM32F407)	Central controller, PID/PI control, CAN BUS
11	Jetson Nano	Onboard computer for ROS2 and image processing
12	USB-CAN TJA1051T	PC/Jetson to CAN BUS communication interface
13	USB-UART	Serial debugging and firmware flashing
14	PS4 Controller	Manual remote control interface



(a) Driver 2 Motor



(b) Driver 4 Motor



(c) Main Board (STM32F407)

Figure 13: Board PCB Design

4.2 Control Speed of DC Motor using PI Controller

4.2.1 DC Motor Modeling

To design a proper speed controller, the DC motor must first be modeled. The motor consists of an electrical part (armature circuit) and a mechanical part (rotor and load), as shown in Figure 14.

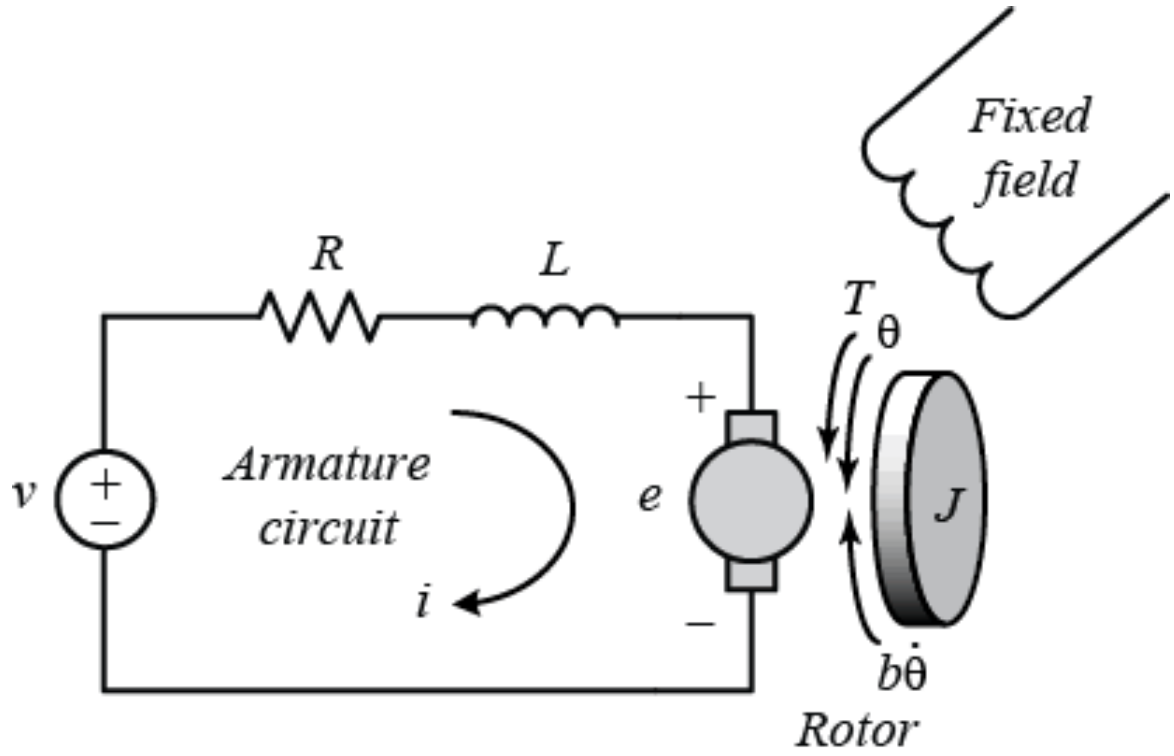


Figure 14: Modeling of DC Motor

Electrical Part:

$$V = L_a \frac{di}{dt} + R_a i_a + K_b \omega \tag{1}$$

Mechanical Part:

$$J \frac{d\dot{\theta}}{dt} + b\dot{\theta} + K_t i_a = T_L \tag{2}$$

Transfer Function:

$$H(s) = \frac{\omega(s)}{V(s)} = \frac{K_t}{(Js + b)(Ls + R) - K_t K_b} \tag{3}$$

where:

- J: moment of inertia of rotor (Kg.m^2)
- R: armature resistance (Ω)
- L: armature inductance (H)
- b: motor viscous friction constant (N.m.s)
- T_L: torque load (N.m)
- $\omega = \dot{\theta}$: angular speed (rad/s)
- K_b: electromotive force constant (V/rad/sec)
- K_t: motor torque constant (N.m/Amp)

Since the electrical time constant $\tau_e = \dfrac{L}{R}$ is much smaller compared to the mechanical time constant, the electrical dynamics can be neglected. The DC motor transfer function can then be simplified to a first-order system:

$$\text{Speed:} \quad H(s) = \frac{\omega(s)}{V(s)} = \frac{K}{\tau s + 1} \tag{4}$$

$$\text{Position:} \quad H(s) = \frac{\omega(s)}{V(s)} = \frac{K}{(\tau s + 1)s} \tag{5}$$

1. **Method 1 Step Response Identification** Transfer function form:

$$H(s) = \frac{\omega(s)}{V(s)} = \frac{K}{\tau s + 1} \tag{6}$$

   - **Step 1:** Input a range of voltages (from 2V to the maximum voltage, in steps) and record the corresponding steady-state speed to obtain a line equation $y = ax + b$, i.e. $(V, \omega(\text{rad/s}))$.
   - **Step 2:** Take the slope $a = K$.
   - **Step 3:** Determine the time constant τ. For a first-order system, the settling time is:

$$T_s = 4\tau \quad \Rightarrow \quad \tau = \frac{4}{T_s} \tag{7}$$

2. **Method 2 Electrical Parameter Identification** This method studies only the electrical part of the motor:

$$V_A = L_a \frac{di}{dt} + R_a i_a + K_B \omega \tag{8}$$

   where L_a, R_a, and K_B are constants.

   At steady state, $\omega_{ss} = \text{const}$ and $i_a = \text{const}$, so $\dfrac{di_a}{dt} = 0$. The equation reduces to:

$$V_A = K_B \omega + R_a i_a \tag{9}$$

   - **Step 1:** Measurable variables are V_A, ω, and i_a.
   - **Step 2:** Input voltage in steps (V_A = 2V : 24V : 2V) though input PWM and measure i_a and ω at each step take read speed from encoder.
   - **Step 3:** Identify R_a and K_B using a least-squares fit. Writing $V_A = K_B\omega + R_a i_a$ for each measurement as $Ax = b$:

$$\begin{bmatrix} \omega_1 & i_1 \\ \omega_2 & i_2 \\ \vdots & \vdots \\ \omega_9 & i_9 \end{bmatrix} \begin{bmatrix} K_B \\ R_a \end{bmatrix} = \begin{bmatrix} V_1 \\ V_2 \\ \vdots \\ V_9 \end{bmatrix} \tag{10}$$

   Then:

$$x = \begin{bmatrix} K_B \\ R_a \end{bmatrix} = (A^T A)^{-1} A^T b \tag{11}$$

Note: Method 2 has not yet been experimentally validated on the actual hardware.

4.2.2 PI Tuning

Once the motor model (gain K and time constant τ) is identified, a PI controller is designed and tuned to regulate the motor speed.

1. **Method 1 Simulink Tuning** The PI controller is tuned interactively in Simulink using the identified plant model, as shown in Figure 15.

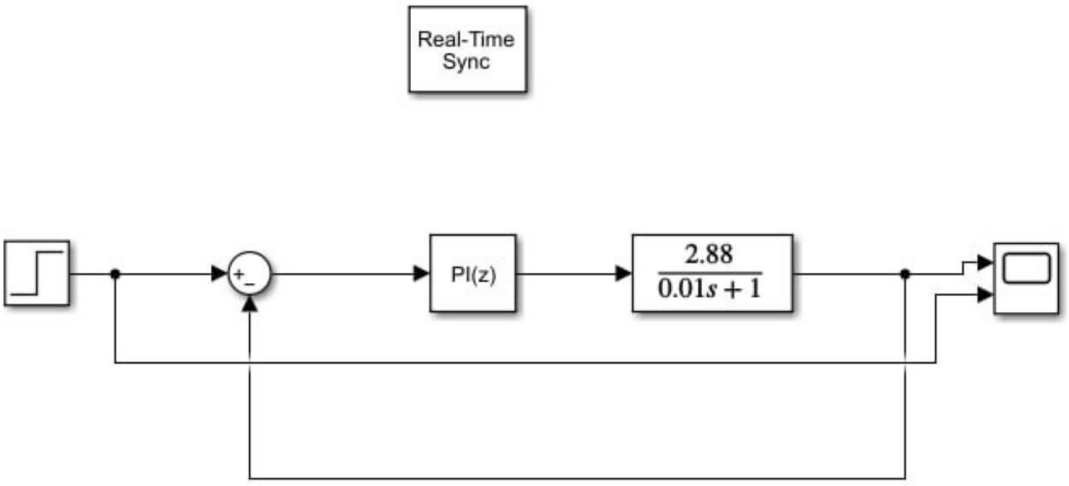


Figure 15: PI Tuning Method 1 on Speed DC Motor in Simulink

Next we go click on PI(z) block to config parameter of auto tuning though we choose desecrate time method and go to output saturation setup limit(1000,-1000):

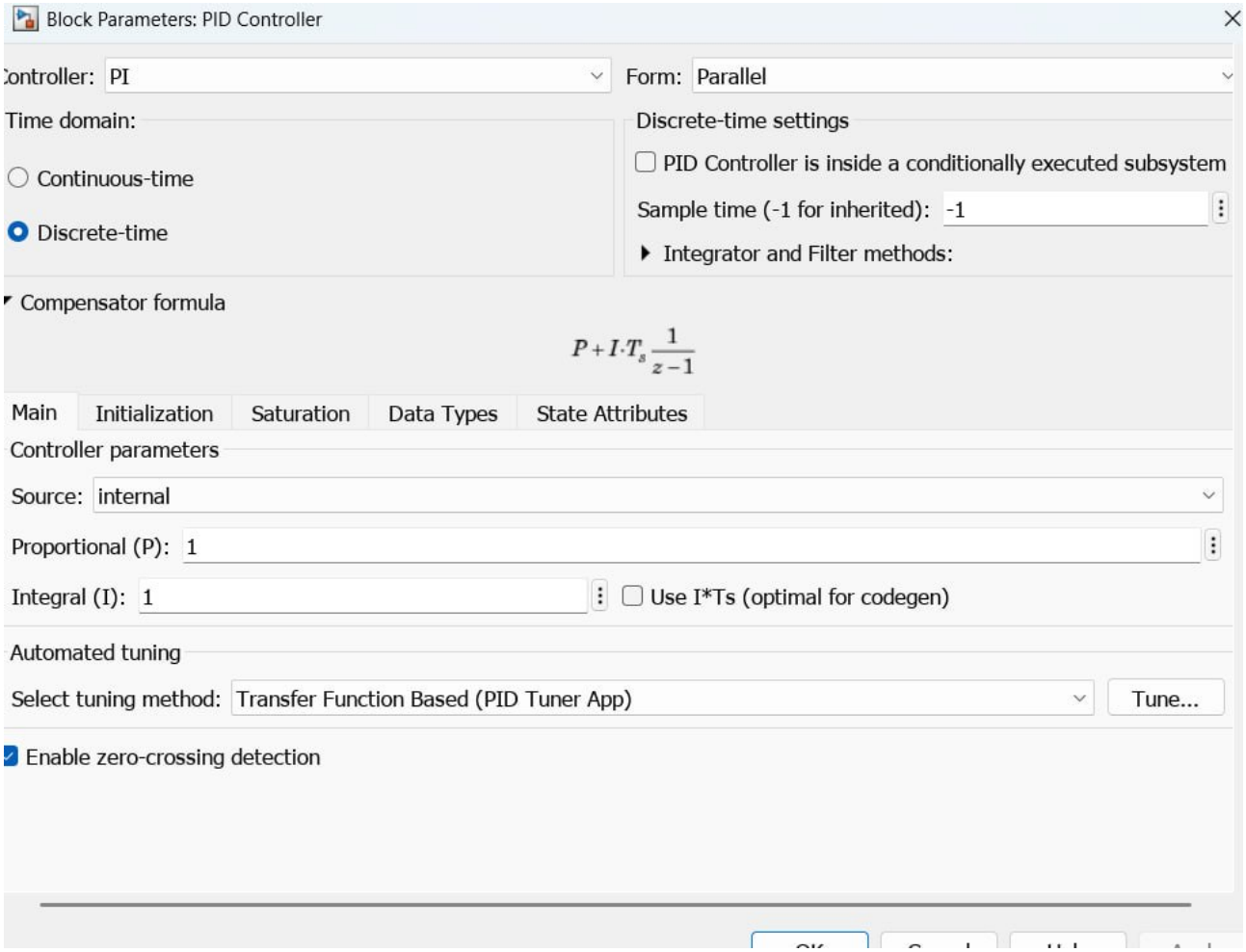


Figure 16: Block PID Paramter Setup

2. **Method 2 MATLAB Script Tuning** Alternatively, the PI gains can be computed directly using a MATLAB script based on the desired settling time and damping ratio, as shown in Figure 17.

```
1
2       % Parameter modeling DC Motor
3 -     K = 3.27;
4 -     tau = 0.044;
5 -     ts = 0.2;  %settling time
6 -     zeta = 0.9    ;
7 -     w = 4/(zeta*ts);
8
9       % CACULATE Kp, Ki
10 -    Kp = (2*zeta*tau*w-1)/K;
11 -    Ki = (tau*w^2)/K;
12
13 -    s = tf('s')
14 -    P = K/(tau*s+1) % Plant
15 -    C = Kp+Ki/s % Control
16
17 -    H = C*P/(1+C*P) % Closed loop transfer function
18 -    minreal(H)
19
20 -    step(H)
21 -    stepinfo(H)
```

Figure 17: PI Tuning Method 2 on Speed DC Motor in MATLAB Script

4.3 P controller with Wheel Robot

This subsection presents the control architecture using a Proportional (P) controller for the outer position loop, combined with a PI-controlled inner velocity loop and EKF-based sensor fusion, as shown in Figure 18.

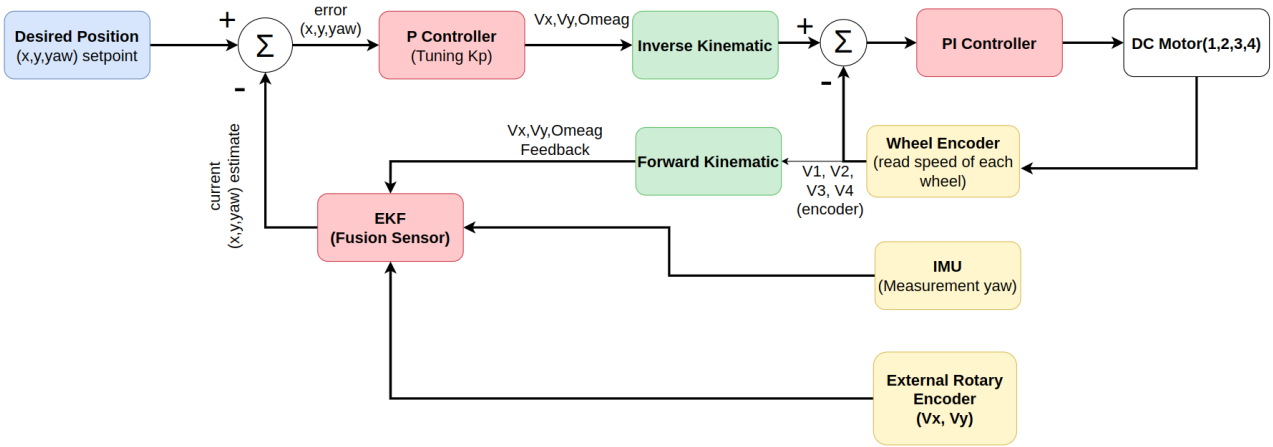


Figure 18: Block diagram of Position Base with P-Controller

The position error $e = [x_d - \hat{x},\ y_d - \hat{y},\ \psi_d - \hat{\psi}]^T$ is regulated by a P controller to generate the desired body velocity $[V_x, V_y, \omega_z]^T$. This is converted to target wheel speeds via Inverse Kinematics, tracked by a PI Controller on each DC motor, and fed back through Forward Kinematics. The EKF fuses wheel odometry, IMU yaw, and external rotary encoder data to estimate the current pose $(\hat{x}, \hat{y}, \hat{\psi})$, closing the outer position loop.

4.4 MPC Controller with Wheel Robot

This subsection describes the alternative architecture where the outer loop uses a trajectory-tracking Model Predictive Controller (MPC) instead of a P controller, as shown in Figure 19.

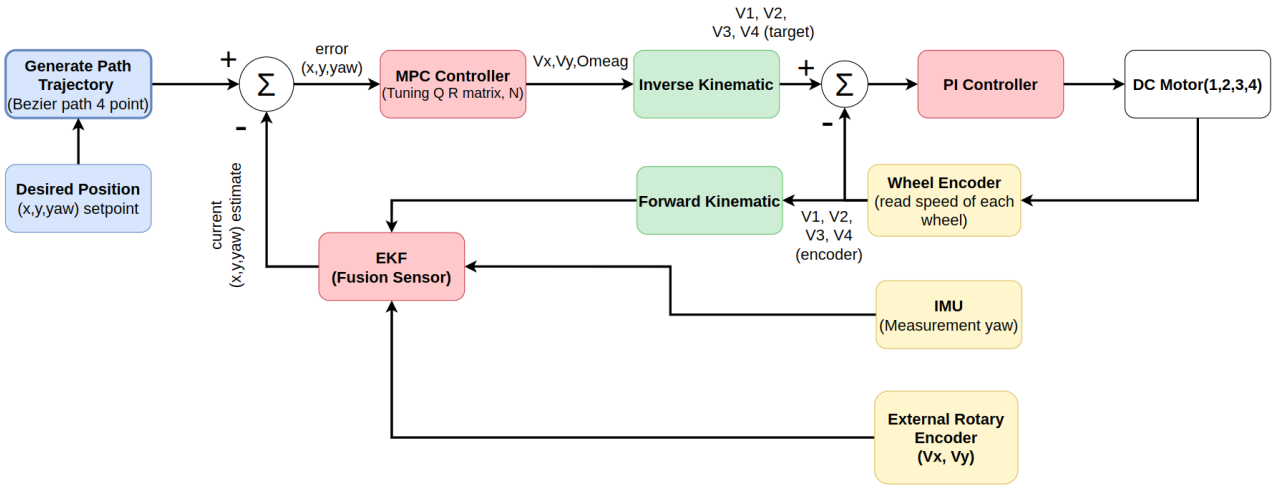


Figure 19: Block diagram of Position Base control with MPC

A Bézier-curve trajectory generator produces a smooth reference path $r_p(k)$ from the desired position. The MPC Controller minimizes the tracking cost $J = \sum_{k=0}^{N-1}(e_k^T Q e_k + \Delta U_k^T R \Delta U_k)$ over a prediction horizon N to compute $[V_x, V_y, \omega_z]^T$. The remaining structure — Inverse Kinematics, PI-controlled motors, Forward Kinematics, and EKF sensor fusion is identical to Section 4.3, isolating the comparison to the outer-loop strategy only.

5 Result and Discussion

5.1 Tuning DC Motor 24V 630RPM in MATLAB

Before identifying the full transfer-function model of the DC Motor 24V 630RPM, the motor's speed response was first characterized using the **PID Tuner app in Simulink**. The test setup consists of a **Step Input** block (start value = 0.0, final value = 1.0, step time = 0.01 s, simulation time = 1.0 s) connected through a summing junction to a **PID Controller** block, whose output drives a **Transfer Function** block representing the identified motor gain ($K = 2.88$). The PID Tuner app was used to automatically compute stabilizing gains for this plant model, as shown in Figure 20.

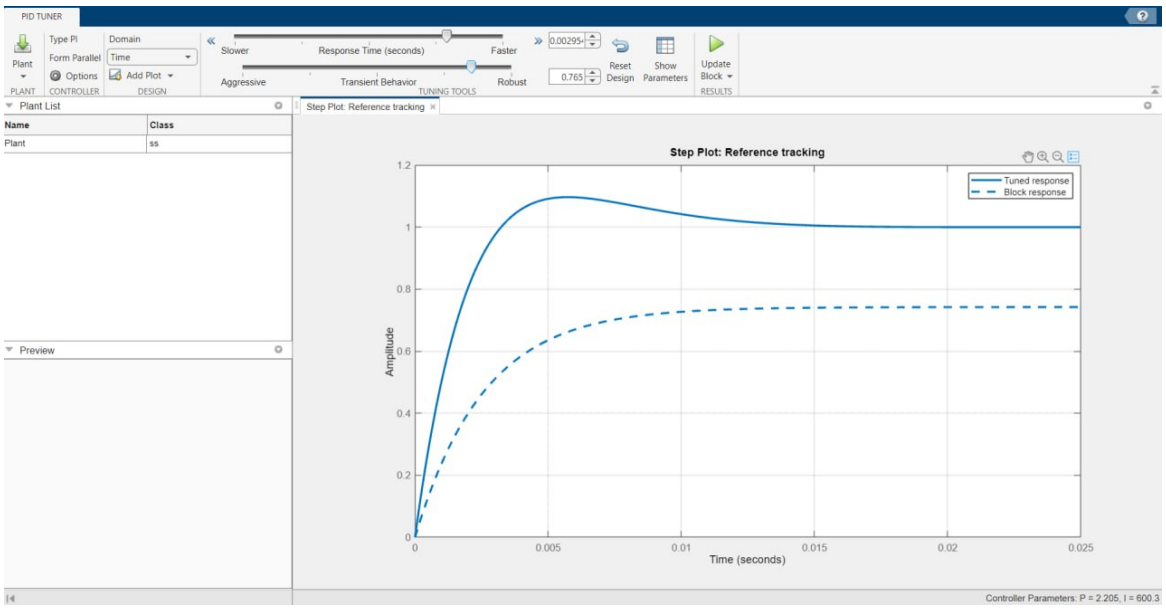


Figure 20: PID Tuner app result for DC Motor plant model ($K = 2.88$): tuned response vs. block response

The tuner produced the controller parameters $K_p = 2.205$ and $K_i = 600.3$ for the plant model. As shown in Figure 20, the *tuned response* (solid line) settles to the reference within approximately 0.015 s with a small overshoot, while the *block response* (dashed line) — corresponding to the default/untuned controller in the Simulink model — is noticeably slower and has not yet reached steady-state within the same window, confirming the tuned gains provide a significant improvement in response speed.

Step Response Verification in Simulink

With the tuned gains applied to the closed-loop PID + motor model, the system was simulated over a 10 s window with a unit step input (Figure 21) to verify the closed-loop tracking performance of the motor speed output (Figure 22).

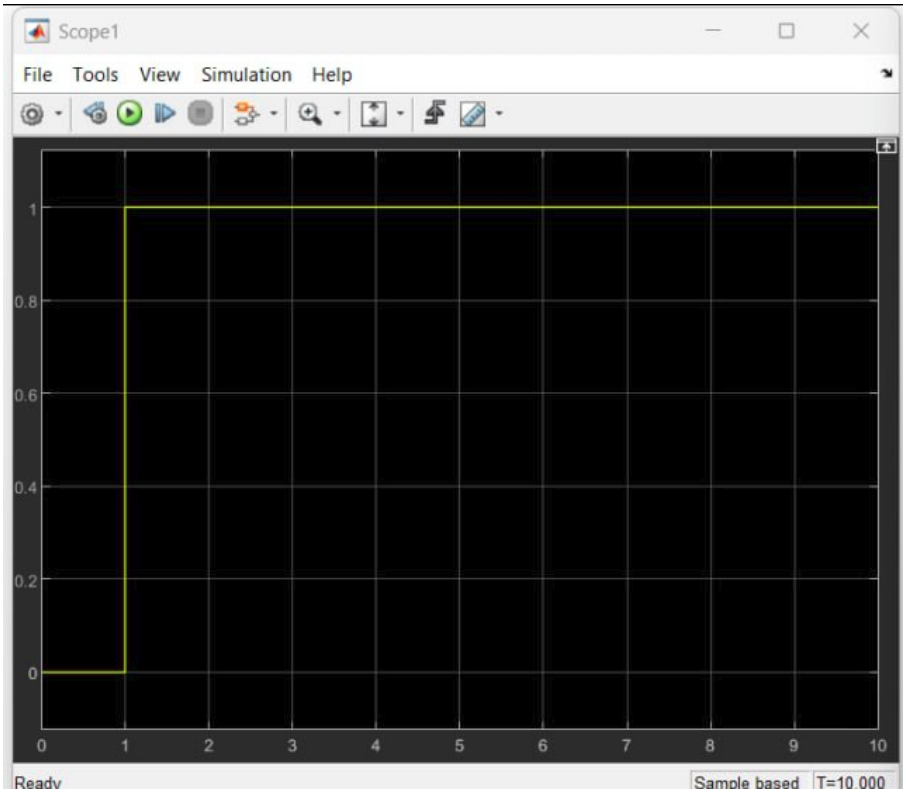


Figure 21: Step input source: ideal reference step from 0 to 1 at $t = 1\,\mathrm{s}$

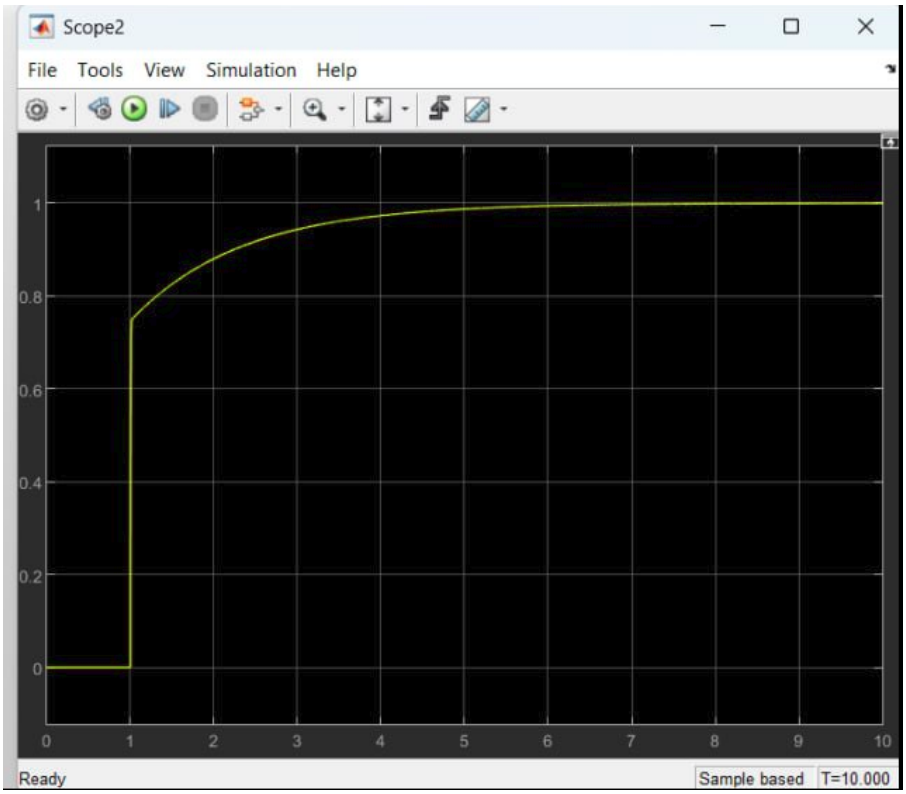


Figure 22: Closed-loop motor speed response to the step input

As seen in Figure 22, the motor output rises sharply from the step onset and smoothly converges to the reference value of 1.0 without oscillation, exhibiting a first-order-like exponential rise consistent with the simplified DC motor transfer function derived in Section **??**. This

confirms that the tuned PI gains ($K_p = 2.205$, $K_i = 600.3$) are suitable starting values for the physical PI speed controller implemented on the STM32 driver board.

Table 3: PID Tuner Result for DC Motor 24V 630RPM Speed Model

Parameter	**Value**
Motor gain K	2.88
Proportional gain K_p	2.205
Integral gain K_i	600.3
Step time	0.01 s
Simulation time	1.0 s (tuner), 10 s (scope verification)

5.2 Simulation LQR with Mecanum Wheel

This subsection presents an offline Python simulation of an **LQR-based trajectory tracking controller** for the Mecanum wheel base, used to validate the gain matrices Q and R before deployment on hardware. The simulation combines a **4-point Bézier** path segment with a **cubic spline** course to generate the reference trajectory, and propagates the robot state using a discrete kinematic model driven by the LQR-computed velocity commands.

Simulation Setup

Physical dimensions of the Mecanum base used in simulation:

- Wheel radius: $R = 76\,\text{mm}$
- Half distance between front/rear wheels: $l_x = 165\,\text{mm}$
- Half distance between left/right wheels: $l_y = 225\,\text{mm}$
- Sampling time: $dt = 10\,\text{ms}$
- Simulation length: 350 steps

At each timestep, the position error $[e_x, e_y, e_\psi]^T$ between the robot's current state and the reference trajectory point is computed and fed into the LQR controller (Section 3.4.1, discrete-time formulation), which returns the optimal body-frame velocity command $U = [V_x, V_y, \omega_z]^T$. This is saturated to the robot's actuator limits, converted to wheel speeds via Mecanum inverse kinematics, then converted back to body velocity via forward kinematics before being applied to the discrete state-space model $X_k = AX_{k-1} + BU_{k-1}$, where B is the yaw-dependent input matrix that rotates body-frame velocity into the global frame.

LQR Gain Tuning

Table 4 lists the state-cost matrix Q and control-cost matrix R used in simulation, tuned to balance fast trajectory convergence against smooth control effort.

Table 4: LQR Cost Matrices used in Simulation

Matrix	**X (or row 1)**	**Y (or row 2)**	**Yaw (or row 3)**
Q (diagonal)	54	23	40
R (diagonal)	0.08	0.08	0.30

Table 5: Actuator and Simulation Constraints

Parameter	Value
Max linear velocity V_x, V_y	$\pm 3.0\,\mathrm{m/s}$
Max angular velocity ω_z	± 1.5708 rad/s
LQR prediction horizon	5 steps

Reference Path

The reference trajectory combines a **4-point Bézier** segment (start pose $(0, 0, 0)$ to end pose $(6.0, -3.4, -1.57)$, offset $= 1.6$) with a **cubic spline** course through a series of fixed waypoints, giving a smooth, continuously differentiable path for the LQR controller to track.

Simulation Result

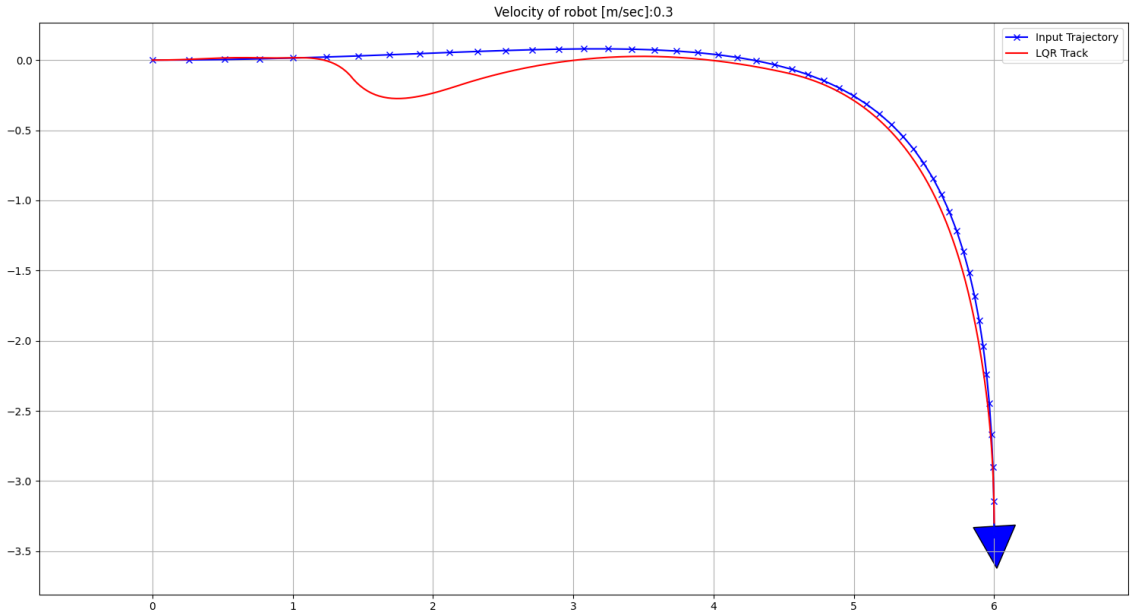


Figure 23: Simulated LQR trajectory tracking on Mecanum wheel base (reference path vs. actual)

Table 6: LQR Simulation Tracking Performance

Axis	Max Error	Final Error	Settling Steps
X (m)	4.676	-0.000	213
Y (m)	3.427	0.026	317
Yaw (rad)	1.338	0.031	N/A

The simulation results confirm the LQR gains produce stable convergence to the reference Bézier/spline path within the actuator saturation limits, providing a validated starting point for gain tuning on the physical hardware.

5.3 Testing Farmer Robot Version 1 with STM32F407VGT6

In this test, MR1 V1 is controlled through an NRF24L01 wireless link, allowing point-to-point navigation and mechanism actions in either Manual or Auto mode.

Physical dimensions of the Mecanum base of MR1 V1:

- Wheel radius: $R = 80\,\text{mm}$
- Half distance between front/rear wheels: $l_x = 700\,\text{mm}$
- Half distance between left/right wheels: $l_y = 600\,\text{mm}$
- Roller angle: 45° to the wheel axle

An STM32F103C8T6 remote board reads joystick/button inputs and sends them via NRF24L01 to the STM32F407VGT6 main board over CAN BUS. In **Manual Mode**, velocity commands are passed directly to the wheels. In **Auto Mode**, the robot's pose (X, Y, θ) is estimated by fusing external rotary encoders, IMU (BNO055) yaw, and laser wall-reference correction, then regulated by a PID Position controller through a sequence of waypoints (finite-state machine).

5.3.1 PID Position Gain Tuning

Table 7: PID Position Controller Gains for MR1 V1

Axis	K_p	K_i	K_d
X	1.4	0.0001	0.004
Y	2.0	0.0001	0.004
Yaw	1.4	0.0001	0.004

Table 8: PID Controller Limits and Sampling Parameters

Parameter	**Value**
Output Max/Min	±0.6
Integral Max/Min (Anti-windup)	±0.6
Derivative Filter α	0.8
Sampling Time T	10 ms
Position Tolerance	0.05 m (0.03 m on precision checkpoints)

5.3.2 Test Result

- **First Robot Position Control** : https://drive.google.com/file/d/18At9AyMkWGuSg9YuErNU3
view?usp=sharing
- **Testing position point to point and yaw of Robot in LAB** : https://drive.google.com/file/d/1LPZjqCaMgB8qClsZEB7OKUkamqHFW8Op/view?usp=sharing

5.4 Testing Farmer Robot Version 2 with ROS2

In this test, MR1 V2 runs on a Jetson Nano under **ROS2 Humble**, using an Omni wheel base. A PS4 controller node publishes velocity and mechanism commands, while a separate PID node handles both manual passthrough and closed-loop Auto positioning.
Physical dimensions of the Omni base of MR1 V2:

- Wheel radius: $R = 50\,\text{mm}$
- Half distance between front/rear wheels: $l_x = 350\,\text{mm}$
- Half distance between left/right wheels: $l_y = 350\,\text{mm}$
- Roller angle: 90° to the wheel axle

5.4.1 System Nodes

The system is split into two ROS2 nodes communicating over topics:

- **PS4 Node:** Reads the `/joy` topic, maps stick axes to V_x, V_y, Ω (scaled by an adjustable speed level from 0–6), and publishes them on `/cmd_vel`. Button presses toggle `/mode` (manual/auto), `/position` (waypoint index), and mechanism states (`Push`, `Catch`, `Drop`, `Shooter`, `Stepper`) on `/concept`.
- **PID Node (Farmer_PID):** Subscribes to `/mode`, `/cmd_vel`, and sensor feedback (`/encoder`, `/current` from IMU/external encoder, `/laser`, `/position`). In **Manual Mode**, it converts V_x, V_y, Ω directly to wheel speeds via Omni inverse kinematics. In **Auto Mode**, it estimates the current pose from encoder odometry (rotated by IMU yaw), regulates it with a PID Position controller, and publishes wheel commands on `/control`.

5.4.2 PID Position Gain Tuning

Table 9: PID Position Controller Gains for MR1 V2

Axis	K_p	K_i	K_d
X	1.5	0.0001	0.004
Y	1.5	0.0001	0.004
Yaw	1.0	0.0001	0.004

Table 10: PID Controller Limits and Sampling Parameters

Parameter	**Value**
Output Max/Min (X, Y)	± 0.5
Output Max/Min (Yaw)	± 0.5
Integral Max/Min (Anti-windup)	± 0.5
Derivative Filter α	0.7
Sampling Time T	10 ms
Laser Correction Tolerance	0.03 m

The robot is sequenced through the planting/harvesting waypoints using a `Position`–`State` finite-state machine: each `Position` index (selected via the PS4 controller) defines a subsequence of goals, with laser feedback used to fine-correct the final approach (± 0.03 m tolerance) before advancing the internal `state` counter.

5.4.3 Video Result

- **Testing Semi-Auto with Concept Collect Rice and Plant Rice** : https://drive.google.com/file/d/1WjebuPmZjyc24y8ZaFsuuYs5xD_2NPVU/view?usp=sharing
- **Testing Ball Shooting Using Laser Calculate Distance** : https://drive.google.com/file/d/1YpoNLd554pL9hlKBa_YQztBFd7nUbozS/view?usp=sharing

5.5 Testing Buffalo Robot with MPC Controller using ROS2

This test evaluates the Buffalo Robot (MR2) using a **Nonlinear MPC controller** (via CasADi) running as a ROS2 node, tracking a reference path generated from cubic spline and 4-point Bézier trajectories instead of a single static setpoint.
Physical dimensions of the Omni base of MR2:

- Wheel radius: $R = 60\,\text{mm}$
- Half distance between front/rear wheels: $l_x = 450\,\text{mm}$
- Half distance between left/right wheels: $l_y = 450\,\text{mm}$
- Roller angle: 90° to the wheel axle

5.5.1 Reference Trajectory Generation

Two path generation methods are used depending on the task segment:

- **Cubic Spline Path:** Used for long-range, multi-waypoint paths (e.g. approaching the game field or retry paths), interpolating a smooth (x, y, ψ, κ) course through a set of fixed anchor points at a fixed step size $ds = 0.1\,\text{m}$.
- **4-Point Bézier Path:** Used for shorter, precision point-to-point moves (e.g. approaching individual silos), generating a smooth curve between the robot's current pose and a target pose with a tunable curvature offset.

At each control step, the closest point on the reference path ahead of the robot is located via a nearest-neighbor search over a fixed look-ahead window (`N_IND_SEARCH`), and the yaw reference is smoothed to avoid $\pm\pi$ wrap-around discontinuities.

5.5.2 MPC Formulation

The controller solves a finite-horizon nonlinear optimal control problem at each timestep using the Omni wheel kinematic model (Section **??**), minimizing the tracking error over a receding horizon and producing the optimal wheel speed sequence $[\omega_1, \omega_2, \omega_3, \omega_4]$. Only the first control input in the optimized sequence is applied at each step, following the standard MPC receding-horizon strategy.

Table 11: MPC Tuning Parameters for MR2 Omni Wheel

Parameter	**Value**
State weight Q (x, y, yaw)	$[1200,\ 1200,\ 800]$
Control weight R (per wheel)	$[1,\ 1,\ 1,\ 1]$
Prediction horizon N	50 steps
Sampling time T	0.1 s
State bounds X_{min}, X_{max}	$[-15, -10, -3.14]$, $[15, 10, 3.14]$
Control bounds U_{min}, U_{max}	-30, 30 rad/s

To reduce abrupt acceleration at the start of each new path segment, the control bounds are ramped up gradually using an exponential speed-up profile $U_{max}(t) = 30\,(1 - e^{-2t})$ until the tracking error norm falls below a threshold, after which full bounds are restored.

5.5.3 Control Architecture

The node operates in two modes, selected via the `/mode` topic:

- **Manual Mode:** Joystick velocity commands (V_x, V_y, Ω) from `/manual` are converted directly to wheel speeds using Omni inverse kinematics.
- **Auto Mode:** The MPC solver computes the optimal wheel speed sequence at 0.1 s intervals based on the current pose feedback (from `/external` and `/encoder`) and the reference trajectory segment selected by the current game state (e.g. `silo1`–`silo5`, `ball`, `retry`).

Wheel velocity commands are published on `/control` at a fixed transmit rate of 10 ms, decoupled from the (slower) 0.1 s MPC solve rate to maintain a smooth command stream to the motor drivers.

5.5.4 Test Result

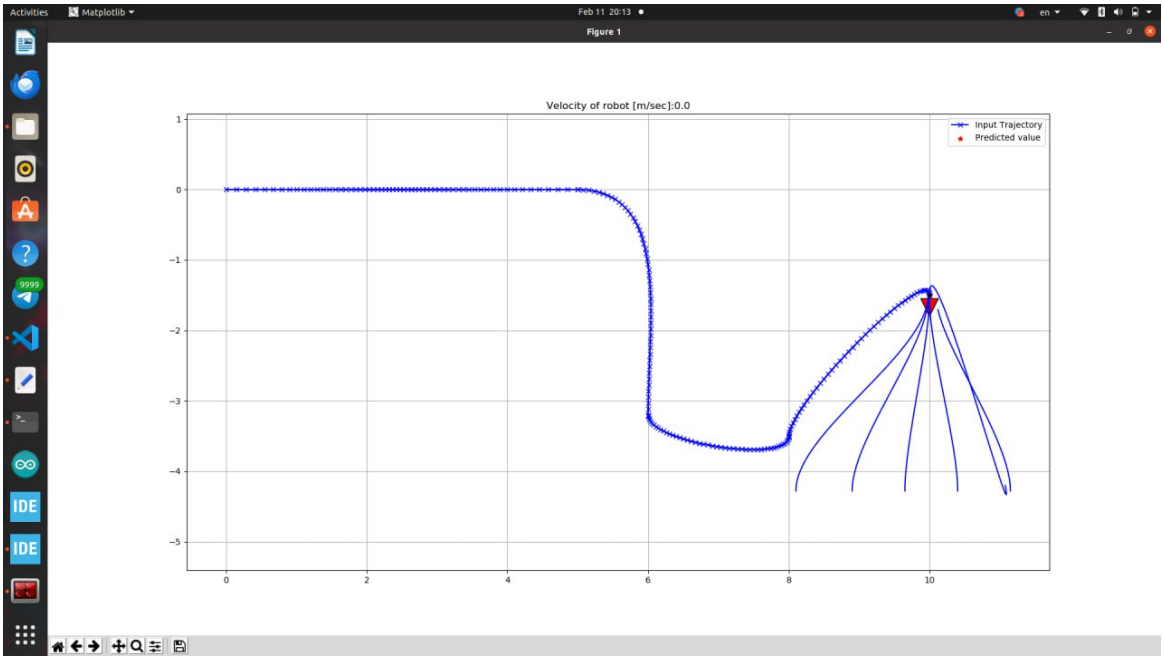


Figure 24: Reference path vs. actual trajectory tracked by MPC (MR2)

Video Result:

- High Speed(V=0.8m/s) = https://drive.google.com/file/d/1UQOuobTfqamGCqyv7MUO5yLniV04
view?usp=sharing
- Middle Speed (V=0.6m/s) = https://drive.google.com/file/d/1kJJ4HOpk1oTqC341G9LARWKSUJ
view?usp=sharing
- Low Speed (V=0.5m/s) = https://drive.google.com/file/d/1VSdVR0gWGB7MZ4bbew3Ozmx8FkkT
view?usp=sharing

5.6 Testing Buffalo Robot with Full Concept Using PID Controller

This test evaluates the complete autonomous behavior of the Buffalo Robot (MR2), integrating navigation, ball detection, ball collection, and silo-sorting into a single closed-loop system running on ROS2. At this stage the robot carries its full mechanism payload (collector, shooter, and silo-sorting hardware), bringing the total robot weight to approximately 25 kg. This added mass and shifted center of gravity made PID tuning considerably more difficult compared to the lighter bare-chassis tests, since the same gains produced noticeably different dynamic response once the mechanisms were mounted.
Physical dimensions of the Omni base of MR2 (same as Section 5.5):

- Wheel radius: $R = 60\,\text{mm}$
- Half distance between front/rear wheels: $l_x = 150\,\text{mm}$ ($L_x = 0.30/2$)
- Half distance between left/right wheels: $l_y = 150\,\text{mm}$ ($L_y = 0.30/2$)
- Roller angle: 90° to the wheel axle

5.6.1 Ball Detection and Position Estimation

Ball detection is performed using an **Intel RealSense D435i** camera on the Jetson Nano, which classifies each detected object by color (*red ball / blue ball / no_ball*) and reports its distance and bearing angle relative to the camera. The ball-detection and classification pipeline itself was developed by a teammate; my contribution was converting this raw detection into a usable **goal position in the robot's base frame**:

$$x_{ball} = d\cos(\gamma), \qquad y_{ball} = d\sin(\gamma) \tag{12}$$

where d is the measured distance and γ is the corrected bearing angle. This local-frame offset is then rotated into the global frame using the robot's current heading ψ and added to the current position estimate:

$$\begin{bmatrix} x_{goal} \\ y_{goal} \end{bmatrix} = \begin{bmatrix} \cos\psi & -\sin\psi \\ \sin\psi & \cos\psi \end{bmatrix} \begin{bmatrix} x_{ball} \\ y_{ball} \end{bmatrix} + \begin{bmatrix} x_{current} \\ y_{current} \end{bmatrix} \tag{13}$$

This global ball position becomes the PID controller's goal setpoint whenever a ball is detected within the designated collection zone, closing the loop between vision detection and motion control.

5.6.2 Ball Collection and Silo Sorting Mechanism

Once the robot approaches the estimated ball position, the collection mechanism (driven by an STM32 board) is triggered. Two **inductive proximity sensors** mounted at the intake confirm successful ball pickup (`ball_already` flag), after which the robot transitions from *search/approach* state to *navigate-to-silo* state. At the silo, laser feedback is used to fine-align the robot's lateral position with the target silo slot, and a second pair of inductive proximity sensors detects whether that silo is already full — if so, the robot automatically advances to the next available silo in the sequence rather than attempting to deposit again.

5.6.3 State Machine Overview

The robot's behavior is governed by a large finite-state machine (`state` variable, 0–20) that sequences through: (1) initial navigation from the starting area to the ball-collection zone via a laser-corrected bridge crossing, (2) ball search including a fallback *scan rotation* routine when no ball is visible for a timeout period, (3) approach and pickup, (4) transit toward the silo area, (5) random silo selection biased by which side of the field the robot entered from, (6) laser-corrected fine alignment and full/empty checking, and (7) return to the ball-collection zone to repeat the cycle. A `silo3` counter tracks how many times the shared middle silo has been used, after which it is excluded from the random selection pool to balance silo usage.

5.6.4 Gain-Scheduled PID Control

Because the tracking accuracy requirement differs sharply between coarse field navigation and millimeter-level silo alignment, the PID gains are **re-instantiated based on the current state** rather than using a single fixed gain set. Table 12 summarizes the gain sets used at each behavioral stage.

Table 12: Gain-Scheduled PID Parameters by Robot State

Behavior Stage	**State Range**	$K_{p,X}$	$K_{p,Y}$	$K_{p,Yaw}$
Ball search / approach	5	0.6	0.6	1.8–2.0
Return-to-silo transit	6–12, 17–18	1.5	1.5	1.8
Silo fine alignment	13–16	1.2	1.2	1.0–1.2

Table 13: Common PID Parameters and Output Limits

Parameter	**Value**
K_i (all axes)	0.0001
K_d (all axes)	0.008
Derivative Filter α	0.7
Sampling Time T	10 ms
Silo Alignment Tolerance	0.04 m (laser-based)
Waypoint Tolerance (transit)	0.1 −0.3 m depending on stage

5.6.5 Test Result

- **Testing position point to point and yaw of Robot in Game Field** : https://drive.google.com/file/d/1D_2T0pVImp7Ewyb6y8Awdn4PKz-fu9rY/view?usp=sharing
- **Testing in a fully game field** : https://drive.google.com/file/d/1Y1ueW9Ad_4-0tj7YG-qGxI7Arwa5AwTZ/view?usp=sharing
- **Testing MR1 and MR2 in real game field to take score** : https://drive.google.com/file/d/1BhOPxmLBL4DmQNeyPgaqTDQ4DhBbj4io/view?usp=sharing

6 Conclusion

This report presented the control system design and testing process for Team ITC01's Farmer Robot (MR1) and Buffalo Robot (MR2) at ABU Robocon 2024. The theoretical foundation covered Mecanum/Omni wheel kinematics, PID, LQR, and NMPC control, and EKF sensor fusion.

On the practical side, MR1 was tested across NRF24L01-based remote control (STM32F407) and ROS2-based teleoperation (Jetson Nano) configurations, while MR2 progressed from isolated position-control tests to a fully integrated autonomous system combining RealSense ball detection, gain-scheduled PID navigation, and proximity-sensor-based ball collection and silo sorting under full mechanical load ($\approx$25 kg). This iterative approach allowed tuning and estimation issues to be identified early, resulting in a validated control architecture for both robots ahead of competition.

References

[1] H. Taheri, B. Qiao, and N. Ghaeminezhad, "Kinematic Model of a Four Mecanum Wheeled Mobile Robot," *International Journal of Computer Applications*, vol. 113, no. 3, pp. 6–9, Mar. 2015. DOI: 10.5120/19804-1586.

[2] University of Michigan, "DC Motor Speed: PID Controller Design," *Control Tutorials for MATLAB and Simulink (CTMS)*. Available: https://ctms.engin.umich.edu/CTMS/index.php?example=MotorSpeed&section=ControlPID

[3] D. U. Rijalusalam and I. Iswanto, "Implementation Kinematics Modeling and Odometry of Four Omni Wheel Mobile Robot on The Trajectory Planning and Motion Control Based Microcontroller," *Journal of Robotics and Control (JRC)*, vol. 2, no. 5, pp. 448–455, Sep. 2021. DOI: 10.18196/jrc.25121.

[4] A. A. Housein, G. Xingyu, W. Li, and Y. Huang, "Extended Kalman Filter Sensor Fusion in Practice for Mobile Robot Localization," *International Journal of Advanced Computer Science and Applications (IJACSA)*, vol. 13, no. 2, 2022. DOI: 10.14569/IJACSA.2022.0130204.

[5] S. H. Żak, "An Introduction to Model Predictive Control," Purdue University, School of Electrical and Computer Engineering, lecture notes. Available: https://engineering.purdue.edu/~zak/Second_ed/MPC_handout.pdf